

Large Language Models As Shannon Lossy Compressors Not Solomonoff Induction Estimators: The Singularity Is Not Near Without Symbolic Model Synthesis

Hector Zenil^{1,2}, Abicumaran Uthamacumaran¹, Luan Ozelim¹

¹ Algocyte | Oxford Immune Algorithmics, Oxford University Innovation
& London Institute for Healthcare Engineering, UK

² Algorithmic Dynamics Lab, Department of Biomedical Computing,
School of Biomedical Engineering and Imaging Sciences, King's Institute for AI,
King's College London, UK

Abstract

On the one hand, the question of whether Large Language Models (LLMs) are Solomonoff induction estimators has become an explicit question at the intersection of Algorithmic Information Theory (AIT) and Machine Learning (ML) of great interest. On the other hand, the now old idea of an AI Singularity that requires a reliable positive-feedback process in which a system can generate, evaluate and retain genuine improvements to itself continues to come up and is a recurrent concept in the discussion of AGI. We connect and provide some answers to these issues based on current assumptions and future developments of neurosymbolic ML. We will demonstrate that cross-entropy, negative log-likelihood and cognate next-token objectives do not or cannot, by themselves, implement Solomonoff induction: they optimise fit to a supplied conditional distribution rather than a program-weighted universal mixture. While more compute within a fixed objective can improve fit without changing the inductive principle, additional computational resources do not intrinsically without external hyper-parameter or architectural changes, behave as optimal predictors in the Solomonoff and Levin sense. While the data-processing inequality (DPI) and Levin non-growth remain valid, we will show that for finite learners and finite observers, theoretical boundaries have less relevancy and generate a drift between possible approaches. To this end, we interpret different resource-bounded estimators as finite tools for mechanism search that show divergence, not violation, of (algorithmic) information conservation laws. A neurosymbolic approach is already being taken and adopted by current frontier-model developers, including models like Fable and Astra, embracing aspects of model synthesis through symbolic computation and cannot longer be considered purely statistical LLMs.

Keywords: large language models; recursive self-training; model collapse; recursive self-improvement; Solomonoff induction; algorithmic probability; cross-entropy; Coding Theorem Method; Block Decomposition Method; Algorithmic Information Dynamics; program synthesis; data-processing inequality, lossy compression; lossless compression, Shannon entropy rate.

1 Introduction

The modern idea of an intelligence explosion begins with a positive-feedback hypothesis. Good's "ultraintelligent machine" was defined by the possibility that a machine capable of improving machine design could initiate a process in which better systems become increasingly effective at producing still better systems [1]. Vinge later placed such a transition at the centre of the technological Singularity, and Kurzweil developed a broader account in which accelerating

technological change eventually produces intelligence beyond ordinary human forecasting [2, 3]. These accounts differ substantially, but they share one structural premise: improvement must feed back into the process that produces further improvement.

The recent success of generative models makes one possible feedback mechanism look unusually simple. A capable model can generate text, code, images, evaluations and synthetic training examples at enormous scale. It can assist in constructing its own training corpus and, increasingly, in writing or modifying parts of the software around it. This invites a tempting identification: if generation can be automated and the generated material can train a successor, perhaps recursive self-training is already the beginning of recursive self-improvement.

A parallel theoretical question has become explicit at the intersection of Algorithmic Information Theory (AIT) and machine learning: whether neural sequence predictors, and Transformers or LLMs in particular, should be understood as approximations to Solomonoff induction. Grau-Moya et al. meta-train neural predictors on sequences generated by Universal Turing Machines in an attempt to amortise a universal predictor into neural networks [16]. Young and Witbrock propose modelling Transformers as bounded approximations to Solomonoff induction [17]. Wan and Mei make the stronger claim that standard LLM loss minimisation computationally approximates the Solomonoff prior and that next-token prediction implements an approximate form of Solomonoff induction [18]. These works make the question precise enough to be tested rather than treated as analogy. The present paper asks which properties follow from the standard likelihood objective itself, which require additional inductive structure, and what changes when program-weighted mechanism search is introduced explicitly.

Wan and Mei’s later formulation describes the next-token connection as a computable surrogate and makes explicit the assumption that the constructed surrogate prior is already close to the Solomonoff prior [19]. The apparent tension with the result developed here is resolved by separating two levels of analysis: what a likelihood objective entails by itself, and what becomes possible when a trained likelihood model is embedded in an additional program-based coding construction.

However, generating more samples from a model does not by itself define what would make a successor better. If the successor is trained only to reproduce the predecessor’s distribution, exact optimisation reproduces that distribution. If finite sampling, filtering or optimisation moves the model away from its predecessor, the movement has no automatic interpretation as improvement. It may be useful, neutral or destructive. A positive-feedback theory therefore needs more than recursion. It needs a directional criterion.

The model-collapse literature, where model collapse denotes degenerative loss or distortion arising across generations of recursively trained generative models, is important because it makes one failure mode of recursive generation visible. Replacement with synthetic data can lose rare events, amplify distortions and change scaling behaviour, while accumulation, replay, correction and other workflows can behave very differently [4, 5, 6, 7, 8, 9, 10, 11, 12]. The lesson is not that synthetic data are intrinsically destructive. It is that the update operator matters. Recursive learning is not one process, and the proportion of synthetic data is not enough to determine its asymptotic behaviour.

The broader issue is rather epistemic. An intelligence explosion is normally imagined as more than a sequence of models that become increasingly self-consistent. The system must somehow improve its representation of the world, its ability to intervene successfully, its algorithms, its scientific theories, its designs or its own architecture. Those improvements require criteria that distinguish a better mechanism from a merely different or more internally coherent one. In science those criteria are supplied by observation, experiment, invariance, prediction and reproducibility. In formal domains they can be supplied by axioms, proof checkers and executable specifications. In engineering they may come from measured performance under independently defined constraints.

This work develops that point while correcting two formulations that are too strong. A

per-generation grounding fraction need not remain bounded away from zero, because a vanishing fraction can still exert unlimited cumulative correction. Algorithmic priors, which favour shorter effective descriptions, also do not escape the data-processing inequality, which limits mutual information under post-processing, or Levin’s algorithmic-information non-growth results, which bound information growth under effective transformations. Their practical relevance lies elsewhere: under finite resources, different representations and search procedures can make different parts of the same available information operationally accessible. Formal information conservation sets a ceiling; it does not choose an empirical method.

The central thesis is therefore narrower than a universal impossibility theorem, but broader than a model-collapse calculation. Closed-loop distribution matching is not itself an improvement operator. The paper also separates the claim that an LLM may learn algorithmic regularities or even implement bounded program-like procedures from the stronger claim that likelihood training itself implements Solomonoff induction. A credible recursive self-improvement architecture must contain a process that proposes candidate changes and evaluates them against criteria not reducible to reproducing the current output distribution. This paper calls that functional requirement *symbolic model synthesis*: the construction, manipulation and testing of explicit generative structures such as programs, equations, causal models, formal rules or executable mechanisms. The term is functional rather than historical. A neural system that genuinely performs these operations counts as a model-synthesis system in the sense intended here.

The distinction is easiest to see by returning to the classical intelligence-explosion argument. Good did not merely imagine a machine producing more artefacts of the same kind. His argument concerned a machine able to improve the design of intelligent machines, including eventually itself. The feedback is therefore second-order: improved capability must improve the process by which further capability is produced. Vinge’s Singularity inherits this recursive structure, while Kurzweil embeds it in a wider argument about accelerating technological change. None of these accounts is reducible to the statement that a sufficiently capable model can manufacture an arbitrarily large training set. The relevant question is whether the system has a reliable operator that maps its present state into a demonstrably better successor and whether improvement of the system also improves that operator.

This distinction matters because several quite different activities are now described informally as “self-improvement”. A language model can generate synthetic examples, critique its own answers, write code, propose changes to prompts, search over tool configurations, or help train a later model. Each may be useful, but none is by definition an improvement step. A generated modification becomes an improvement only relative to some criterion, and the criterion itself must remain meaningful as the system changes. If the same model family supplies the proposal, the target and the judgement, a positive score can reflect increasing internal agreement rather than increasing competence with respect to the world or to an independently specified task.

There is therefore an important asymmetry between candidate production and candidate validation. Modern generative models have made candidate production extraordinarily cheap. They can propose code, proofs, explanations, molecular structures, experimental protocols and model architectures at a scale that was previously impossible. This changes the economics of search, but it makes validation more, not less, important. Once proposals become abundant, the bottleneck moves towards deciding which proposals survive contact with constraints that are not simply restatements of the generator’s current preferences. A theory of recursive self-improvement has to account for this second half of the loop.

The point can also be expressed in terms of error correction. Human scientific and engineering institutions are not reliable because every individual proposal is well grounded. They are reliable, when they are reliable, because conjectures can be rejected by measurements, failed replications, incompatible observations, formal contradictions or engineering tests. An

autonomous machine need not reproduce those institutions, but it needs some functional analogue if it is to improve open-endedly rather than merely become more self-consistent. The central issue is thus not whether humans remain present, but whether the system retains channels through which its own current model can be proved wrong.

2 Preliminaries and notation

Model collapse refers here to degenerative loss or distortion that can arise in recursively trained generative models, including loss of support or diversity under replacement-style workflows; the precise behaviour depends on the update rule and data-retention scheme [4, 5, 6, 8]. *Grounding* denotes corrective influence supplied by information or constraints not reducible to neutral reproduction of the current model distribution. In the simple mixture model below it is represented by an external data weight, but elsewhere it can also arise from retained evidence, environmental interaction, experiments, formal verification or invariant task criteria. We use the term *epistemic openness* for the broader architectural property of remaining answerable to such non-circular correction.

The statistical notation is standard [13]. Upper-case letters such as P , Q_t , and R denote probability distributions unless otherwise stated. P will usually denote a fixed external or reference distribution, Q_t the model distribution at recursive step t , and R a candidate successor distribution. Lower-case q_t denotes a probability vector on a finite alphabet. The quantity N_t denotes the sample size used at step t . The parameter $\alpha_t \in [0, 1]$ denotes the weight assigned to the external reference distribution in the simple grounded mixture model, and $\beta_t = \prod_{s < t} (1 - \alpha_s)$ denotes the residual multiplicative influence of the initial discrepancy after t updates. For a discrete distribution q , Shannon entropy is $H(q) = -\sum_i q_i \log q_i$, while $D_{\text{KL}}(p||q)$ denotes Kullback–Leibler divergence. Mutual information between random variables X and Y is denoted by $I(X; Y)$. The base of the logarithm is immaterial to the qualitative arguments; base 2 is used when information is interpreted in bits.

An autoregressive LLM represents a token sequence $x_{1:n}$ by the conditional factorisation

$$p_\theta(x_{1:n}) = \prod_{t=1}^n p_\theta(x_t \mid x_{<t}), \quad (1)$$

where $x_{<t}$ is the preceding context and θ denotes the model parameters [14, 15]. Thus the probability assigned to a complete sequence is the product of the probabilities assigned to its successive tokens conditional on the preceding context, which is the standard autoregressive next-token factorisation. For a finite vocabulary $\mathcal{V}$, a decoder produces a real-valued logit $z_\theta(v \mid x_{<t})$ for each token $v \in \mathcal{V}$, and a softmax converts these logits into a conditional next-token distribution,

$$p_\theta(v \mid x_{<t}) = \frac{\exp z_\theta(v \mid x_{<t})}{\sum_{u \in \mathcal{V}} \exp z_\theta(u \mid x_{<t})}. \quad (2)$$

The normalisation maps the unconstrained logits to non-negative probabilities that sum to one across the vocabulary, with larger logits receiving a larger share of the next-token mass. Generation then selects or samples the next token from this conditional distribution. Standard autoregressive pre-training commonly minimises token-level negative log-likelihood, equivalently cross-entropy or maximum-likelihood loss,

$$\mathcal{L}_{\text{NLL}}(\theta) = -\mathbb{E}_{X \sim P} \left[\sum_t \log p_\theta(X_t \mid X_{<t}) \right]. \quad (3)$$

The loss is large when the model assigns low probability to the token that actually occurs, so minimisation rewards progressively better agreement with the conditional token distribution

represented in the training data. Perplexity is an exponential transformation of average negative log-likelihood and therefore preserves the same model ordering for a fixed evaluation set. We refer below to cross-entropy, negative log-likelihood, maximum likelihood and their direct equivalents as *likelihood-based next-token objectives*. These definitions concern the predictive interface and optimisation criterion; they do not assume that a neural network is incapable of learning an internal algorithm or structured representation.

The *data-processing inequality* (DPI) states that if random variables form a Markov chain $X \rightarrow Y \rightarrow Z$, then $I(X; Z) \leq I(X; Y)$ [13]. In this paper DPI is used only as an information-conservation constraint: post-processing cannot create additional mutual information about an external variable beyond that contained in the inputs to the processing chain. Later, M denotes an unknown mechanism, D the observations available to a learner, B fixed background information and Y the learner’s output. The paper also introduces T_b for the transcript obtainable under a finite inspection budget b , $\mathcal{A}_b$ for a class of procedures available under computational budget b , and I_b^{acc} for the corresponding task-relevant accessible information. The last quantity is an operational construction introduced here, not a replacement for standard mutual information.

The algorithmic-information terminology follows the standard framework of Solomonoff induction, Kolmogorov complexity and Levin’s algorithmic probability [20, 21, 22, 23]. Let U be a fixed universal prefix machine and p a self-delimiting program. The prefix Kolmogorov complexity of a finite object x is $K_U(x) = \min\{|p| : U(p) = x\}$, defined up to the usual additive constant associated with the choice of universal machine. Algorithmic probability assigns greater weight to outputs generated by shorter program s ; schematically, $m_U(x) = \sum_{p: U(p)=x} 2^{-|p|}$. The coding theorem relates these quantities by $K_U(x) = -\log_2 m_U(x) + O(1)$. Exact Kolmogorov complexity and the universal semimeasure are uncomputable. Levin’s information non-growth results are algorithmic counterparts of the same conservation intuition: under the conditions of the theorem, effective processing cannot systematically manufacture unbounded algorithmic information that was absent from the relevant inputs [21, 22]. Nothing in this paper treats these results as false or violated; the issue examined is what finite procedures can make operationally accessible under bounded resources.

The *Coding Theorem Method* (CTM) is a finite computational method that estimates algorithmic probability and hence algorithmic complexity by enumerating or systematically running classes of small programs or machines and measuring their output frequencies [24]. CTM is therefore an approximation within a specified finite machine and runtime regime based on or inspired by the universal distribution.

The *Block Decomposition Method* (BDM) is a greedy algorithm that extends CTM-style estimates to larger objects by decomposing them into smaller components, reusing local algorithmic-complexity estimates and accounting for repeated components [25] penalising by repetitions according to classical information theory hence combining the best of both classical and algorithmic worlds. Its estimates consequently depend to some controlled degree on the chosen decomposition [25] as well as on the underlying CTM values for a reference machine, shown to in general produce convergent values (hence possibly not much dependant on reference machine as long as it is ‘natural’) [24] and, in the worse case, converging to Shannon entropy rate in the limit [25].

Algorithmic Information Dynamics (AID) studies changes in estimated algorithmic information under controlled perturbations in order to analyse structural contribution, dynamical organisation and reprogram mability [26, 29]. AID is used here as a perturbational and mechanism-oriented framework, not as a claim that algorithmic complexity alone identifies causal truth.

A *generative mechanism* means a program, equation, causal model, simulator or other rule capable of producing or explaining the observations and of entailing consequences beyond simple replay of the observed sample. Different mechanisms may be observationally compatible,

so observational fit alone need not identify a unique causal structure; causal identification requires additional assumptions, interventions or other identifying information depending on the problem [28]. The term *symbolic model synthesis* is used in a functional sense for constructing, manipulating and testing explicit generative structures whose consequences can be checked against evidence or formal constraints. It does not require classical symbolic AI or a separate symbolic module. A neural or hybrid system that performs these functions counts as a model-synthesis system in the sense used here. Large language model (LLM), artificial general intelligence (AGI) and artificial superintelligence (ASI) are used in their conventional broad senses; no particular operational definition of AGI or ASI is required by the mathematical results below.

3 Recursive self-improvement and the autonomy premise

The Singularity argument is often described in terms of autonomy, but autonomy has several meanings that should not be conflated. A system may become operationally autonomous because it needs less direct human labour. It may become computationally autonomous because it writes code, allocates resources or schedules its own searches. Neither form of autonomy requires epistemic isolation. An autonomous scientific agent could continuously read instruments, query databases, run experiments, interact with an environment and submit conjectures to independent verifiers. Such a system may require little human intervention while remaining strongly grounded in information that does not originate from its own current model distribution.

The relevant distinction is therefore between *autonomy* and *endogenisation*. A recursive loop becomes epistemically closed to the extent that its training targets, evaluators and evidence are generated from the same endogenous model family without an independent correctness signal. This is the regime in which self-consistency can increasingly substitute for external correction. It is also the regime that makes model collapse relevant to arguments about recursive self-improvement. The problem is not that humans have left the loop. The problem is that the world, an invariant evaluator or another non-circular source of constraint has effectively left the loop.

This distinction changes how the original grounding parameter should be interpreted. If α_t measures the proportion of samples drawn directly from an external distribution, then a vanishing α_t does not by itself imply loss of grounding. The absolute amount of new evidence may still grow, old evidence may be retained, or a vanishing proportion may have sufficient cumulative influence. Moreover, some forms of grounding cannot naturally be represented as a sample fraction at all. A proof checker, a conservation law, an executable specification or an experimental outcome can constrain an update even when almost all training tokens are synthetic.

It is useful to distinguish three generic loops. In a closed distributional loop, the current model generates the examples and the successor is trained primarily to match the resulting distribution. In an externally evaluated loop, the model may still generate most candidate data or actions, but a rule set, environment, benchmark, verifier or measurement supplies an independent selection criterion. In a mechanism-seeking loop, the system constructs an explicit generative hypothesis, derives consequences that were not simply replayed from the training sample, and uses observation or formal checking to discriminate among hypotheses. These loops can share the same neural components while having very different epistemic structures.

Self-play illustrates why the distinction matters. A chess or Go system can generate essentially all of its own game trajectories, yet the rules and the win condition remain fixed outside the current policy distribution. Recursive generation is therefore not closed in the relevant sense. Likewise, code-generation systems can improve measured runtime if execution

on a trusted benchmark supplies the score, and theorem-proving systems can generate their own conjectures and candidate proofs if a sound checker supplies correctness. These examples show that synthetic generation is compatible with improvement when something independent decides what counts as success.

The difficult case for broad recursive self-improvement is that the evaluator itself may need to improve. A fixed chess score defines a bounded task. An intelligence explosion, by contrast, is usually taken to involve expanding competence across domains, improving representations, discovering new objectives or subgoals, and modifying the machinery that performs evaluation. The more of this stack becomes endogenous, the more important it becomes to ask what prevents the system from optimising its own proxy, narrowing its hypothesis space or mistaking internal agreement for external correctness.

This is the reason the paper retains the language of the Singularity. The issue is not whether a future system can be autonomous. It is whether the positive-feedback loop assumed by Good, Vinge and Kurzweil has been specified at the epistemic level. Recursive generation provides an abundant source of candidate material. It does not, on its own, supply the criterion that turns recursive change into recursive improvement.

This way of framing autonomy also clarifies why self-play and formal reasoning are not counterexamples to the argument. In self-play, most of the trajectories may be generated internally, but the game rules and terminal conditions provide a stable external structure. In formal mathematics, a model may generate both conjectures and candidate proofs, but a proof checker can reject an invalid derivation without consulting the model’s confidence. In program optimisation, the model can propose its own rewrites while execution time, memory use or test-suite correctness provide measurements that are not determined by the proposal distribution. The proportion of human-authored material can approach zero in all three cases without producing epistemic closure.

Open-ended scientific discovery is harder because the relevant criteria are not always fixed in advance. A system investigating an unfamiliar physical process may need to decide which variables to measure, which instruments to build, which interventions distinguish competing mechanisms and which apparent regularities are artefacts of measurement. Here improvement cannot be reduced to maximising a static benchmark. The learner must participate in constructing the very tests that make its hypotheses answerable to the world. This is one reason the paper treats model synthesis and independent evaluation as coupled operations rather than as two permanently separate modules.

The same distinction prevents an overly simple reading of “external grounding”. External information is not synonymous with human text. A sensor reading, a failed experiment, a theorem checker, an executable simulator constrained by known physics, a database measurement or an interaction with another agent can all alter the evidential situation. Conversely, a large corpus of nominally real data can provide little corrective value if it is redundant with what the model already represents. The mathematically convenient parameter α_t captures one kind of mixture process, but epistemic openness is a broader architectural property.

4 Recursive self-training: identity, grounding and collapse

Generative models are naturally described by probability distributions, whether explicitly or implicitly. An autoregressive language model assigns conditional probabilities to token sequences; a diffusion model defines a generative process through iterative denoising; a GAN defines a distribution through the output of its generator. These architectures differ, but the recursive-training question can be isolated at a higher level: what happens when the distribution produced by one generation increasingly determines the training target of the next?

Let Q be a model distribution and suppose a successor distribution R is trained, in the

population limit, by minimising log-loss on samples from Q . The expected loss is

$$\mathbb{E}_{X \sim Q}[-\log R(X)] = H(Q) + D_{\text{KL}}(Q \| R). \quad (4)$$

Here $H(Q)$ is fixed by the target distribution, whereas the KL-divergence term measures the discrepancy between the successor R and Q and is the only part that can be reduced by changing R . Since this term is non-negative and vanishes when $R = Q$, exact self-training has the optimum

$$R^* = Q. \quad (5)$$

At the population level, the best possible successor under this objective is therefore the distribution it is trained to imitate: pure self-imitation is an identity operation expressing fidelity rather than an intrinsic direction of improvement. This observation is more important for recursive self-improvement than a universal collapse theorem would be. If the objective is only to reproduce Q_t , the exact optimum is Q_t . If another objective moves a successor in a useful direction, that objective has entered the effective learning system and should be analysed as part of the loop.

Grounding can be represented in a simple population model by mixing a fixed reference distribution P with the current model distribution Q_t :

$$Q_{t+1} = \alpha_t P + (1 - \alpha_t) Q_t, \quad 0 \leq \alpha_t \leq 1. \quad (6)$$

Here α_t is the fraction of the update attributable to the external reference distribution at generation t , while $1 - \alpha_t$ is the retained contribution from the current model. The update is therefore a weighted average of external correction and self-retention. Subtracting P and iterating gives

$$Q_t - P = \beta_t (Q_0 - P), \quad \beta_t = \prod_{s=0}^{t-1} (1 - \alpha_s). \quad (7)$$

Thus the discrepancy from P retains its direction but is scaled by β_t , which accumulates every preceding retention factor. The asymptotic behaviour is therefore governed by the whole grounding schedule through the cumulative product β_t , not by the pointwise limit of α_t . If no α_t is exactly one, β_t tends to zero when the cumulative grounding is sufficient. Under the usual infinite-product criterion, this occurs when

$$\sum_{t=0}^{\infty} \alpha_t = \infty. \quad (8)$$

The condition allows individual corrections to become arbitrarily small while requiring their cumulative influence not to remain finite; under the stated infinite-product conditions this is sufficient for the residual factor β_t to vanish. A grounding fraction can consequently vanish while continuing to remove the initial discrepancy. For example, $\alpha_t = 1/(t+2)$ tends to zero but gives $\beta_t = 1/(t+1)$, so Q_t converges to P . By contrast, $\alpha_t = 1/(t+2)^2$ also tends to zero, but the cumulative product remains positive and a residual contribution from Q_0 survives. A condition such as $\inf_t \alpha_t > 0$ is therefore sufficient in this model but not necessary.

This calculation also clarifies what the grounding parameter represents and, equally importantly, what it does not. The instantaneous value of α_t measures the contribution of the external distribution at a particular stage, but it does not by itself determine whether the process remains grounded. The relevant quantity in this model is the cumulative influence of the sequence $\{\alpha_t\}$, as captured by β_t . More generally, even this cumulative mixture is only one mathematical representation of grounding. Experiments, environmental interaction, formal

verification, invariant constraints and independently defined performance criteria can all provide corrective information without being naturally expressible as a fraction of training samples. The mixture model therefore illustrates a broader progression from instantaneous grounding, to cumulative grounding, and ultimately to epistemic openness as an architectural property. For recursive self-improvement, the important question is not whether α_t remains bounded away from zero, but whether the system preserves channels through which its hypotheses, modifications and evaluators remain answerable to constraints that are not simply regenerated from its own current distribution.

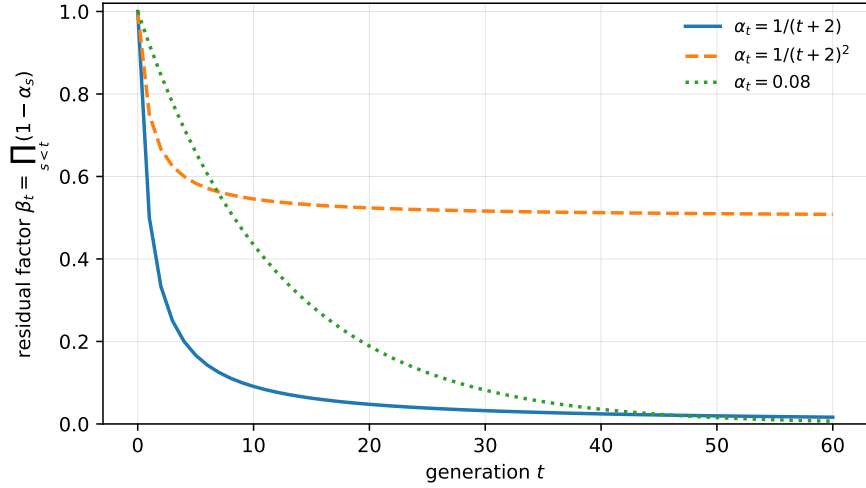


Figure 1: Three grounding schedules/regimes and the corresponding residual factor β_t . A vanishing per-generation fraction does not determine the asymptotic result. The cumulative product does.

Finite sampling introduces a different effect. On a finite alphabet, let q_t be the current probability vector, draw N_t samples from it, and let the empirical frequencies become q_{t+1} . Define the Gini–Simpson diversity by

$$V(q) = 1 - \sum_i q_i^2. \quad (9)$$

This quantity vanishes when all probability mass is concentrated on one outcome and grows as the mass is spread across several outcomes, making it a simple diversity measure for the finite-alphabet model. For multinomial resampling,

$$\mathbb{E}[V(q_{t+1}) \mid q_t] = \left(1 - \frac{1}{N_t}\right) V(q_t). \quad (10)$$

Because $1 - 1/N_t < 1$ for every finite sample size, each replacement step removes on average a fraction $1/N_t$ of the current diversity, with the effect diminishing as N_t grows. Repeated finite replacement therefore reduces expected diversity whenever N_t is finite. With a fixed finite sample size, the process eventually concentrates on a random surviving state. The mechanism is straightforward: finite resampling introduces sampling fluctuations, and making the empirical sample the complete target of the next generation repeatedly converts those fluctuations into the new population.

This result is intentionally narrow. It describes neutral multinomial replacement, not neural optimisation in general. Its value is that it isolates an exact source of degeneracy: repeated finite resampling without a restoring target. It also shows why a universal state-independent

entropy decrement is too strong. Near a point mass the expected change can be arbitrarily small, and at a point mass it is zero.

Grounding and sampling must therefore be analysed together. In the same finite-alphabet model, a useful sufficient condition for mean-square consistency is

$$\sum_{t=0}^{\infty} \alpha_t = \infty, \quad N_t \alpha_t \longrightarrow \infty. \quad (11)$$

The two requirements play distinct roles: the first supplies unbounded cumulative correction towards the reference distribution, while the second makes finite-sampling noise negligible relative to that correction. Together they are sufficient for the mean-square error to vanish in this model, yet neither requires the grounding fraction itself to remain bounded away from zero. For example, the schedule $\alpha_t = 1/(t+2)$ and $N_t = (t+2)^2$ satisfies both conditions even though α_t vanishes.

These simple results accord with the broader model-collapse literature once different workflows are kept separate. Recursive replacement can produce collapse or tail loss [4, 5, 6, 11]; mixed-data analyses can control error under explicit conditions [7]; correction functions can stabilise self-consuming loops [9]; and accumulation or retention can prevent the degradation seen under replacement [8, 10]. Token-level editing is another example in which the data-generation rule is altered rather than left as pure self-replacement [12]. These results are not contradictory. They concern different update operators.

The conclusion required here is consequently more general than any statement about the limiting value of α_t . Exact self-imitation has no improvement direction. Finite recursive replacement can lose diversity. Retained evidence, correction, curation and external objectives can change the dynamics. A model that perfectly reproduces its predecessor has achieved fidelity, not improvement; a model that moves away from its predecessor because of sampling noise has achieved change, not necessarily useful novelty; and a model whose successors are selected by evidence or an independent criterion is participating in a different process altogether. For recursive self-improvement, the decisive question is therefore what corrective information and evaluation remain available to the loop, rather than the proportion of synthetic data considered in isolation.

5 Information conservation and empirical openness

The data-processing inequality is a theorem about random variables under a specified joint distribution. Let M denote an unknown mechanism, D the observations available to a learner, B fixed background information, and Y the output of a procedure using only (D, B) and internal randomness independent of M once (D, B) are given. Then

$$I(M; Y) \leq I(M; D, B). \quad (12)$$

The bound states that processing the observations and background information cannot make the output Y contain more mutual information about the unknown mechanism M than was present in those inputs. It constrains information flow without prescribing an efficient inference procedure. Nothing in this work contradicts that inequality [13]. Levin’s non-growth results make an analogous point for algorithmic information under computable and certain random transformations [21, 22]. A fixed computation cannot manufacture information about an external mechanism from nothing.

The methodological question is different. Real inference is conducted with local and partial access to data, finite computation, finite memory, restricted representations and bounded search. The fact that all procedures lie below the same information ceiling does not tell us

which procedure will actually discover a useful regularity or mechanism. Data processing is therefore a foundational starting point, not a method-selection theorem.

A simple example makes the distinction explicit. Let M be uniformly distributed over n positions and let the fixed data object D be a one-hot vector whose single 1 occurs at position M . Globally, D identifies M completely. A learner that can inspect only the first b coordinates has less operational access. If the 1 is found, the index is known; if it is not found, M remains uniformly distributed over the uninspected positions. The mutual information in the finite query transcript is

$$I(M; T_b) = \log_2 n - \frac{n-b}{n} \log_2(n-b). \quad (13)$$

For this example the expression quantifies exactly how much the learner has discovered after inspecting b coordinates: it is zero at $b = 0$, increases with the inspection budget, and reaches the full $\log_2 n$ bits when all positions are accessible, even though the underlying data object and the global information ceiling never change. No information law is violated. More of the fixed information has become operationally accessible to the bounded procedure.

The same idea can be stated more generally. Let $\mathcal{A}_b$ be a nested class of task-relevant procedures available under computational budget b , with outputs restricted to the kind of prediction, mechanism description or decision under study. Define

$$I_b^{\text{acc}}(M; D, B) = \sup_{A \in \mathcal{A}_b} I(M; A(D, B)). \quad (14)$$

The supremum represents the best task-relevant information about M that any procedure available within budget b can extract from the fixed evidence (D, B) . If $\mathcal{A}_b \subseteq \mathcal{A}_{b+1}$, then

$$I_b^{\text{acc}}(M; D, B) \leq I_{b+1}^{\text{acc}}(M; D, B) \leq I(M; D, B). \quad (15)$$

The inequalities make the resource distinction explicit. Enlarging the available procedure class cannot reduce the best accessible information, but no such procedure can exceed the information contained in the complete inputs; additional computation can therefore improve access without violating the global ceiling. This is not proposed as a new universal information measure. It is an operational device for separating a global information ceiling from what a specified class of finite procedures can actually make usable for a task.

This distinction is especially important when applying Levin's theorem to open-ended algorithmic search. An entire schedule of increasingly large searches can indeed be described by one meta-algorithm, and information non-growth applies to the whole computation. But describing an unlimited search does not make the result of an unperformed search available at an earlier finite stage. Computability in principle and accessibility under a finite resource budget are different methodological facts. The theory bounds information; it does not erase computational depth.

An empirical system can therefore remain open-ended in two different senses. Changing representation, search depth or hypothesis class can increase computational access to consequences already implicit in fixed data. Observation, intervention or independent verification can instead supply genuinely new empirical input. The former changes accessibility without changing the global information in the original observations; the latter changes the information set itself. Scientific inference uses both.

This is the intended position of CTM relative to DPI and Levin non-growth. CTM does not escape either theorem, and a universal prior is not a permanent stream of empirical grounding about the particular mechanism under study. The claim is methodological: two learners may satisfy the same information-conservation bound yet differ substantially in what structure they can expose, manipulate and test under finite resources. Information conservation remains true while being largely non-discriminating between empirical methods operating below the same ceiling.

The information-theoretic issue is sometimes made to carry more methodological weight than it can support. The data-processing inequality answers a precise question about information flow under a specified probabilistic dependence. Levin-style non-growth results answer a related algorithmic question about the inability of effective processing to manufacture unlimited algorithmic information from nothing. Both are indispensable boundary conditions. Neither says that all computable procedures presented with the same finite observations are equally useful, equally efficient or equally capable of finding a compact explanation within a finite time.

This distinction is familiar in ordinary computation. The fact that the output of a program is determined by its input does not imply that the output is already available to a bounded observer in any operational sense. Factoring a large integer, solving a difficult search problem or running a long simulation may reveal consequences of a fixed input that were always implicit in it, yet obtaining those consequences can require substantial computation. In empirical inference the situation is richer because the choice of representation also determines which regularities are cheap to express and search. A bound on total information does not erase these differences in computational accessibility.

For this reason, “open-ended” has two meanings in the present argument. A learner can be computationally open-ended by enlarging the class of hypotheses it can search, the depth of programs it can execute, or the representations it can construct. This can expose consequences already implicit in its existing evidence. A learner can also be empirically open by acquiring new measurements, interventions or independently verified outcomes. The first changes access to information without violating conservation; the second changes the evidential input itself. Scientific practice continually alternates between the two. A theory suggests a calculation or experiment, the result changes the evidence, and the revised evidence motivates another theory.

This is also why a universal prior should not be described as though it were a permanent stream of observations about the particular world being studied. Its role is structural. It changes the ordering of candidate explanations by favouring shorter generative descriptions. Whether that bias is useful depends on the relation between the chosen description language, the available computational budget and the actual mechanism. The empirical advantage, when there is one, lies in reaching a useful hypothesis with finite resources, not in violating a global information bound.

The distinction has a direct methodological consequence. An objection based only on DPI or non-growth cannot decide between a distributional estimator, a causal-search procedure, a theorem prover and a program -synthesis system if all operate on the same evidence. They all respect the relevant conservation law. What matters empirically is which consequences they can derive, which hypotheses they can represent, how much computation they require and whether their outputs survive independent tests. The information bound is the floor of the discussion, not its conclusion.

6 Mechanism discovery, causality and algorithmic model synthesis

The original motivation for introducing symbolic and algorithmic methods was to distinguish distribution fitting from mechanism discovery. That distinction should not be stated as a claim that neural networks are incapable of representing mechanisms, nor as a claim that a particular loss function is intrinsically incapable of causal reasoning. Neural systems can implement algorithms, learn structured latent variables, call tools and participate in experimental design. The narrower point is that fitting an observational distribution does not, by itself, identify which causal or generative mechanism produced it. Different mechanisms can induce the same observational distribution, and causal conclusions require additional assumptions, interventions or other identifying structure [28].

This matters for recursive self-improvement because prediction and explanation can come apart. A model can reproduce the statistical regularities of its training environment extremely well while remaining unable to distinguish among mechanisms that agree on observed data but diverge under intervention, distribution shift or extrapolation. Conversely, a compact mechanism can make predictions outside the observed support, but only if the mechanism has been identified well enough to justify those predictions. The issue is not correlation versus causation as a slogan. It is the logical gap between matching observations and selecting among observationally compatible generators.

Kant’s distinction between analytic and synthetic judgements provides a useful historical analogy, but only an analogy [27]. The paper does not claim that LLMs are literally confined to Kantian analytic judgements, nor that a neural network cannot generate a novel proposition. The relevant contrast is between rearranging or extending consequences within an accepted representational frame and acquiring a warranted new model of how the world works. For the purposes of recursive self-improvement, what matters is not novelty of wording but whether a proposed law, mechanism or design survives a test whose outcome is not fixed by the model’s own current beliefs.

Algorithmic probability offers one principled way to organise a search over generative explanations. For a universal prefix machine U , the Solomonoff–Levin semimeasure can be written schematically as

$$m(x) = \sum_{p:U(p)=x} 2^{-|p|}, \quad (16)$$

where each program p that generates x contributes weight $2^{-|p|}$, so shorter generating programs contribute exponentially more mass. Solomonoff–Levin induction thereby organises prediction around a mixture of possible generative mechanisms rather than observed frequency alone, with the coding theorem linking high algorithmic probability to low Kolmogorov complexity up to the usual machine-dependent additive constant [20, 21, 22, 23]. Exact Kolmogorov complexity and the full universal distribution are uncomputable, so every practical implementation is necessarily finite and approximate.

The motivation for introducing algorithmic probability is not that statistical regularity is unimportant. It is that statistical similarity and generative simplicity answer different questions. Two observations can be far apart under a surface metric and nevertheless be generated by the same short program ; conversely, a distribution can be fitted accurately by many mechanisms that disagree sharply outside the observed support. A generative description is valuable when it compresses the observations by specifying a process that can be run, modified and interrogated.

Solomonoff’s construction provides an idealised formal version of this preference. programs that generate an observation contribute probability mass weighted by their description length. The resulting universal distribution is not computable, and this limitation is fundamental rather than an implementation inconvenience. Its relevance here is therefore methodological: it defines a principled direction for searching over generative explanations, while practical methods must approximate that direction within finite program classes and runtime budgets.

The distinction becomes especially concrete at the next-token interface. For a fixed data-generating distribution, expected token log-loss is minimised by the corresponding conditional data distribution whenever that distribution is representable by the model. This is the conditional analogue of the identity calculation in the previous section: cross-entropy can be written as entropy plus a KL divergence, and its optimum is obtained by matching the target conditional distribution. Negative log-likelihood and maximum likelihood express the same optimisation, while perplexity is a monotone transformation of the resulting average log-loss. These objectives can produce extremely capable predictors, but the objective itself contains no term that enumerates candidate generating programs or weights them by $2^{-|p|}$.

This yields a limited but sharp conclusion. A standard autoregressive LLM trained by cross-

entropy or a cognate likelihood-based next-token objective is not, by virtue of that objective alone, a Solomonoff predictor. The softmax in (2) normalises scores over vocabulary tokens, whereas Solomonoff induction assigns predictive weight through a mixture over programs that can generate compatible continuations. Low next-token log-loss therefore does not imply that the predictor is estimating the universal algorithmic semimeasure. This is not a representational impossibility theorem for transformers: a neural network could in principle learn or implement a Solomonoff-like bounded procedure. The point is that such an algorithmic prior or program-search operation is additional structure, not something entailed or identified by the likelihood objective itself.

6.1 Solomonoff induction and the location of the inductive burden

The distinction above bears directly on recent attempts to interpret neural predictors as approximations to Solomonoff induction [16, 17, 18]. The relevant issue is not whether a neural architecture can represent algorithmic procedures. It is where the structural assumptions that determine successful induction enter the effective learning system. Cross-entropy is a scoring rule rather than an architecture, but the estimator obtained by minimising it depends on the hypothesis class, representation and effective probability mass presented during training. A useful way to expose this dependence is to formalise curation as a transformation of the training distribution.

For simplicity, let P be a discrete observational distribution and let $w(x) \geq 0$ represent selection, filtering, oversampling, curriculum weighting or another transformation of the probability mass supplied to the learner. Provided $0 < Z_w < \infty$, define

$$P_w(x) = \frac{w(x)P(x)}{Z_w}, \quad Z_w = \mathbb{E}_{X \sim P}[w(X)]. \quad (17)$$

Here $w(x)$ changes the relative mass assigned to examples and Z_w renormalises those weights, so P_w remains a probability distribution. A likelihood-based estimator trained on this effective distribution minimises

$$\mathcal{L}_w(Q) = \mathbb{E}_{X \sim P_w}[-\log Q(X)] = H(P_w) + D_{\text{KL}}(P_w \| Q). \quad (18)$$

The decomposition is the usual cross-entropy identity applied to the curated distribution P_w : once the effective training mass has been reshaped, the likelihood objective rewards agreement with that reshaped distribution. In an unrestricted population model the optimum is therefore

$$Q_w^* = P_w. \quad (19)$$

With no restriction on the model family, the minimum is reached by reproducing P_w exactly. The likelihood objective therefore inherits the curation embodied in w rather than determining for itself how the original distribution should have been reweighted. In particular, the loss does not determine the weighting function w . If an external process changes which observations are retained, how frequently they are sampled, or which synthetic examples are admitted, it has changed the target distribution whose probability mass the estimator is asked to reproduce. This may be entirely legitimate, but the structural work performed by that process should be counted as part of the effective learning architecture rather than attributed to cross-entropy itself.

Architecture and representation can be made explicit by letting $\mathcal{Q}_A$ denote the family of distributions representable under an architecture or hypothesis class A . When the minimum is attained, a population solution is a KL projection

$$Q_{A,w}^* \in \arg \min_{Q \in \mathcal{Q}_A} D_{\text{KL}}(P_w \| Q). \quad (20)$$

When the architecture can represent only distributions in $\mathcal{Q}_A$, optimisation selects a member of that class minimising the KL discrepancy from P_w . The realised estimator is therefore jointly determined by the observational distribution P , its external transformation w , and the representational class $\mathcal{Q}_A$. A sufficiently expressive likelihood-trained system can exhibit sophisticated inductive behaviour, but the objective alone still specifies neither the architecture nor how the training mass is constructed. The Universal-Turing-Machine data used by Grau-Moya et al. to meta-learn universal prediction strategies is a particularly direct illustration: the resulting neural learner is trained by ordinary optimisation, while a program-generating process supplies the algorithmically structured meta-training distribution [16].

One can quantify the magnitude of a distributional intervention, without pretending to measure human effort or semantic quality, by

$$C_{\text{mass}}(w) = D_{\text{KL}}(P_w \| P). \quad (21)$$

Here the KL divergence records only how far the effective training distribution has been displaced from the observational distribution. It is deliberately not a universal measure of curation cost, human labour, intelligence or semantic quality. Its purpose is to separate improvement caused by changing the estimator from improvement caused by changing the probability mass that the estimator is asked to fit.

The role of computational power can be separated in the same way. Suppose additional computation is used only to solve a fixed likelihood-learning problem more accurately. If $\hat{Q}_{A,w,c}$ denotes the result obtained under computational budget c , its optimisation error can be represented schematically by

$$\mathcal{L}_w(\hat{Q}_{A,w,c}) - \inf_{Q \in \mathcal{Q}_A} \mathcal{L}_w(Q) \leq \varepsilon_c, \quad \varepsilon_c \rightarrow 0. \quad (22)$$

The left-hand side is the remaining optimisation gap at computational budget c . The limit $\varepsilon_c \rightarrow 0$ is not asserted as a universal convergence theorem for neural optimisation; it isolates the case in which additional computation solves the same fixed likelihood problem increasingly well, without by itself changing the target distribution, hypothesis class or inductive principle. Holding P_w , $\mathcal{Q}_A$ and the objective fixed, reducing ε_c approaches the same optimum more accurately. It does not introduce a Solomonoff mixture over program s . If additional computation is instead used to enlarge the model class, then $\mathcal{Q}_A$ changes, but that is an architectural expansion. It still does not follow from the likelihood objective alone that the enlarged family is ordered according to algorithmic probability.

The stronger recent claim that loss minimisation itself should be interpreted as program-length optimisation therefore requires assumptions beyond the cross-entropy identity [18]. The decomposition in (18) fixes the population optimum from the supplied distribution; it contains no explicit $2^{-|p|}$ weighting and no program enumeration. A trained network may nevertheless internalise compression, algorithms or a bounded approximation to universal prediction. The distinction here is between what a particular trained system may implement and what its likelihood objective mathematically entails.

Wan and Mei’s construction is usefully read as complementary to this distinction rather than as a contradiction of it. Starting from a trained LLM, they combine the model with an arithmetic encoding of the observed string, decoding information, a random seed and a prefix-code construction on a universal machine. This defines a computable family of programs and a corresponding surrogate mass, which may be written schematically as

$$\overline{M}_\theta(x) = \sum_s 2^{-\ell(\bar{f}_\theta(x,s))} \leq M(x). \quad (23)$$

The surrogate $\overline{M}_\theta$ sums the algorithmic weights of the explicit program family constructed from the trained model; the inequality records that this computable family contributes only part of

the universal program mass $M(x)$, which also includes all other valid generating programs. Wan and Mei then relate reduction of language-model loss to shortening the arithmetic description of x , thereby increasing the algorithmic mass contributed by this particular computable program family [18]. Nothing in the cross-entropy calculation above excludes such a construction. A likelihood-trained model can be used as a compression component inside a larger algorithmic program family, and improving that model can improve the resulting surrogate.

The refinement concerns what follows from this construction. The inequality $\overline{M}_\theta(x) \leq M(x)$ establishes that the constructed programs contribute legitimate mass within a Solomonoff-style universal mixture, but an increasing lower bound is not by itself a convergence statement for the full universal semimeasure. In particular, reducing the code length generated by one model-dependent program family does not alone establish that the remaining Solomonoff mass from all other programs becomes negligible. Wan and Mei’s later formulation makes the corresponding condition explicit at inference time: their surrogate next-token interpretation is stated under the assumption that $M(x_{1:t}) \approx \overline{M}(x_{1:t})$ [19]. Under such an assumption, the program-based surrogate can yield a normalised continuation distribution closely related to the LLM’s next-token probabilities. The substantive algorithmic step is therefore the construction of $\overline{M}$, together with the assumption that it adequately approximates M , rather than the cross-entropy identity alone.

The two results consequently operate at different levels. Equation (18) is an objective-level statement: for a supplied target distribution, unrestricted population log-loss is minimised by matching that distribution. Wan and Mei provide a construction-level statement: a trained likelihood model can be wrapped in explicit coding and universal-machine machinery so that its code lengths define a computable subset of Solomonoff-style program mass. Read this way, their result refines rather than contradicts the present argument. It supplies a concrete example of the additional algorithmic structure through which a likelihood-trained LLM may participate in a Solomonoff-style estimator, while the present analysis identifies why that conclusion is not entailed by the entropy-based objective in isolation. This distinction also clarifies the role of CTM below: CTM makes the program-space approximation explicit by estimating algorithmic probability from enumerated computational mechanisms rather than deriving the program family only from the code lengths of a particular trained predictor.

6.2 Why conventional compression is a weak test of algorithmic induction

A further methodological issue concerns the use of conventional compression algorithms, such as LZW, or entropy-optimised neural predictors as evidence for a “simplicity bias” related to Algorithmic Information Theory. Such comparisons can be informative about statistical redundancy, but they should not be interpreted as strong evidence for approximation to algorithmic probability or Solomonoff induction.

LZW is a lossless dictionary-based compressor. Its success depends on discovering repeated substrings and exploiting regularities that reduce empirical description length. In this sense, it certainly exhibits a form of simplicity preference: sequences with repeated or statistically regular structure are compressed more effectively than sequences lacking such redundancy. However, this is not the same object as algorithmic probability.

For a reference universal prefix machine U , Solomonoff–Levin algorithmic probability is defined schematically by

$$m_U(x) = \sum_{p:U(p)=x} 2^{-|p|},$$

where the probability assigned to x arises from the aggregate weight of all programs p on U that generate it, with each program weighted exponentially by its description length. Solomonoff induction therefore organises prediction over a universal generative hypothesis space. LZW does not enumerate such programs, does not weight hypotheses by universal program length, and does not operate over a universal mixture of computable generators.

The distinction is therefore stronger than the observation that both compression and algorithmic probability reward “simplicity”. A conventional compressor can prefer statistically simple strings without approximating Kolmogorov complexity in the algorithmic sense. Repetition, low entropy rate, short-period structure, local dependencies, or regularity within a restricted grammar can all make an object easy to compress for LZW while saying little about the shortest program or family of programs capable of generating the object.

In particular, the Shannon entropy rate of a stationary source,

$$h = \lim_{n \rightarrow \infty} \frac{1}{n} H(X_1, \dots, X_n),$$

characterises asymptotic statistical compressibility under suitable conditions. A compressor that asymptotically approaches this rate demonstrates successful exploitation of statistical redundancy. But this fact alone does not imply approximation to $K_U(x)$ or to $m_U(x)$. Algorithmic probability contains ordinary statistical compressibility as a special and comparatively weak case: a regular source may be statistically compressible because of repetition or predictable frequencies, whereas the distinctive content of AIT concerns the length and distribution of generative programs capable of producing the data.

This creates a potential circularity when an entropy-based or dictionary-based compressor is used to validate another entropy-optimised predictor. If one method is sensitive to statistical redundancy and another is evaluated by how well it reproduces the same redundancy, agreement may largely establish that both systems detect compressible statistical structure. In that sense, “compression indicates simplicity” and “simplicity predicts compression” can become two formulations of the same entropic fact. Such an experiment may verify a necessary condition for Solomonoff-style induction, because any sufficiently general predictor should exploit statistically visible regularity, but the condition is too weak to identify the specifically algorithmic content of Solomonoff induction.

The same issue applies to neural predictors trained with likelihood or cross-entropy objectives. Their loss can be written as

$$H(P, Q) = H(P) + D_{KL}(P \| Q),$$

so optimisation rewards agreement with the supplied predictive distribution. A neural model may learn rich internal structure and may even implement algorithmic procedures, but low cross-entropy or strong predictive compression does not by itself establish that the model is assigning mass according to $2^{-|p|}$ over a universal program space. Predictive success is therefore compatible with, but not diagnostic of, Solomonoff-like induction.

The distinction becomes sharper when stated in terms of computational grammar. Choosing a black-box compressor does not remove the choice of computational formalism; it merely makes that choice implicit. A particular compressor, finite-state predictor, or neural architecture operates within the effective grammar and hypothesis class permitted by its implementation. If that computational system is not universal in the relevant sense, it is closer to selecting a particular restricted Turing machine than to selecting a reference Universal Turing Machine. Opacity does not confer universality.

This is important because replacing an explicit reference UTM by an arbitrary black-box predictor is sometimes presented as avoiding the arbitrariness of reference-machine choice. It does not. The black box still embodies a specific computational language, architecture, representation and inductive bias. The difference is that these restrictions are less explicit. A non-universal predictor may therefore be more restrictive than an explicitly chosen UTM, not more general.

By contrast, a UTM-based construction has a principled theoretical status. Although algorithmic complexity depends on the choice of reference UTM up to the usual additive constant, all universal machines share the ability to represent the full class of computable functions. For universal machines U and V , the invariance theorem gives

$$K_U(x) \leq K_V(x) + c_{UV},$$

for some machine-dependent constant c_{UV} independent of x . The practical importance

of the constant can be substantial for short objects, but this is fundamentally different from restricting inference to a computational class that is not universal at all.

CTM-style methods make this distinction explicit. Rather than using compression performance as an indirect proxy for simplicity, they approximate algorithmic probability by enumerating or systematically sampling computational mechanisms within a specified universal formalism and measuring their output frequencies. In a finite approximation with program-length and runtime bounds L and T , one may write

$$m_{U,L,T}(x) = \sum_{\substack{|p| \leq L \\ U(p)=x \text{ within } T}} 2^{-|p|}.$$

As the accessible program and runtime classes are enlarged, the approximation ranges over an increasingly rich subset of the program space of the reference UTM. Because exact algorithmic probability is lower semicomputable rather than computable, this does not provide an effective finite procedure for obtaining the exact universal semimeasure. It does, however, provide a principled limiting connection to a well-defined algorithmic object that an arbitrary statistical compressor does not acquire merely through improved predictive performance.

Moreover, the reference-machine issue should not be caricatured as the arbitrary selection of a single UTM. Previous CTM work has examined multiple universal computational formalisms and machine classes and has studied empirical stability, convergence and robustness of resulting complexity rankings across reference choices. The relevant comparison is therefore not between an “arbitrary UTM” and a supposedly assumption-free black box. It is between an explicitly specified universal program space, whose machine dependence can be studied, and an implicit computational grammar whose restrictions may be harder to characterise and which may not be universal.

For these reasons, compressors such as LZW, entropy-rate estimators, and likelihood-trained neural predictors are useful statistical controls but weak tests of Solomonoff-like induction. Stronger evidence requires experiments in which statistical compressibility is controlled while program-level generative structure varies. Ideally, one would compare sequences with similar entropy rate, local redundancy, or conventional compression ratio but different algorithmic generative complexity, and then ask whether the predictor follows the program-induced conditional distribution rather than merely the statistical surface.

This distinction is also supported by the empirical results in Section 7. There, the CTW baseline achieved substantially better next-bit coding than the Transformer while failing to improve out-of-support extrapolation or exact mechanism recovery. Better predictive compression therefore did not imply better access to the generating mechanism. This separates ordinary coding performance from explicit mechanism-space inference and illustrates why entropic compression, while necessary for broad predictive competence, is too weak by itself to establish approximation to algorithmic probability or Solomonoff induction.

Returning to the broader data–computation trade-off, the distinction can now be located more precisely.

If a system learns to select, reweight or generate its own training data and validates those transformations against independent evidence, that curation process becomes part of the inference architecture. In that case it would be misleading to describe the complete system as merely a cross-entropy estimator. This point strengthens rather than weakens the architectural thesis: likelihood objectives can participate in a Solomonoff-style or mechanism-seeking system, but the additional search, curation and validation machinery must be included when explaining where the inductive power comes from.

The Coding Theorem Method (CTM) approaches that finite regime by enumerating large classes of small programs and estimating algorithmic probability from their output frequencies [24]. BDM extends the strategy to larger objects by decomposing them into smaller components for which CTM estimates can be reused [25]. Algorithmic Information Dynamics uses related estimates under controlled perturbations to study candidate causal contribution,

dynamical structure and reprogram mability [26, 29]. None of these approximations is a causal oracle. Their relevance here is that they search a space organised by generative description rather than only by similarity in an observed sample.

The three algorithmic tools play different roles in this argument and should not be conflated. CTM is the most direct connection to algorithmic probability. By exhaustively or systematically running small computational devices, it estimates the frequency with which finite outputs are generated and thereby supplies empirical approximations to algorithmic probability for objects small enough to be covered by the chosen machine space. Its value is not that the approximation is universal in practice, but that it provides a non-Shannon estimate sensitive to candidate generative structure.

BDM addresses the immediate scaling problem. Exact CTM tables necessarily cover only relatively small objects. BDM decomposes a larger object into parts for which algorithmic-complexity estimates are available and combines those estimates while accounting for repeated components. This introduces decomposition dependence and cannot reproduce the full Kolmogorov complexity of an arbitrary large system, but it provides a practical bridge between local algorithmic estimates and larger structured data. In the present paper BDM is enabling rather than foundational: it shows how the mechanism-oriented bias need not remain confined to toy strings.

Once a candidate object or mechanism has been represented, the question for self-improvement is no longer only what compact process could generate this object but also which components matter to its behaviour, and what changes when they are perturbed or intervened? AID uses changes in estimated algorithmic information under controlled perturbations as one way to study structural contribution and reprogram mability. This is closer to the intervention side of the self-improvement problem. A mechanism that can be represented but not meaningfully modified is of limited use to an architecture expected to redesign itself or its models.

The distinction is important because none of these methods supplies truth automatically. CTM can privilege a short explanation that happens to be wrong. BDM can obscure dependencies that cross a chosen decomposition. AID can assign significance differently under different representations or perturbation schemes. Their role is therefore to propose and organise structural hypotheses that can be executed or perturbed. Epistemic work is completed only when the consequences of these hypotheses are compared with independent evidence or constraints.

The difference can be illustrated with a familiar toy example. From the finite sequence 2, 4, 6, 8, a program such as “generate successive even numbers” implies 10, 12, ... even though those values were not observed. The inference is not guaranteed to be true: infinitely many other programs agree with four observations and diverge later. The example nevertheless captures the function of model synthesis. A generative hypothesis can entail unobserved consequences. Those consequences then become predictions to be checked. A purely empirical replacement distribution containing only the observed values has no comparable entailment unless additional structure has been learned or supplied.

The finite computational character of this search is central. If program length and runtime are bounded, define

$$m_{L,T}(x) = \sum_{|p| \leq L: U(p)=x \text{ within } T \text{ steps}} 2^{-|p|}. \quad (24)$$

This finite version of algorithmic probability counts only programs no longer than L that halt within T steps. Increasing either bound can admit candidate generators that were previously computationally inaccessible and therefore cannot reduce the finite mass. A candidate mechanism excluded by an earlier computational budget can become accessible later, while the observations remain unchanged. This is exactly the resource-bounded situation discussed in the previous section. No information is created; the search has made a different part of the

hypothesis space operationally available.

The same construction can be expressed as a bounded next-token predictor. Let $M_{L,T}^{\text{pre}}(x)$ denote the algorithmic mass of programs of length at most L whose outputs begin with the prefix x within T steps,

$$M_{L,T}^{\text{pre}}(x) = \sum_{\substack{|p| \leq L: U(p) \text{ outputs a string with prefix } x \\ \text{within } T \text{ steps}}} 2^{-|p|}. \quad (25)$$

The programs need not halt exactly after producing x ; it is enough that their outputs begin with the observed prefix. The quantity therefore measures finite algorithmic support for continuations compatible with the current context. For a next token $a \in \mathcal{V}$, this induces the finite continuation distribution

$$\pi_{L,T}(a \mid x) = \frac{M_{L,T}^{\text{pre}}(xa)}{\sum_{b \in \mathcal{V}} M_{L,T}^{\text{pre}}(xb)}, \quad (26)$$

whenever the denominator is positive. The normalisation compares the program-weighted mass supporting each possible continuation xa across the vocabulary. Unlike the learned softmax in (2), the scores in (26) are explicitly determined by the generating programs admitted by the bounded search. This is still not exact Solomonoff induction: the program length, runtime and machine class are finite, and the universal semimeasure remains uncomputable. It nevertheless makes explicit what a resource-bounded Solomonoff-style next-token estimator would have to add beyond likelihood-based token prediction.

A CTM-style enumeration supplies finite empirical approximations to algorithmic probability in precisely this generative direction. An open-ended implementation can progressively enlarge the enumerated machine or program classes and runtime budgets, thereby expanding the set of generative explanations that are operationally available rather than merely refining parameters inside a fixed token-level objective. BDM can make related algorithmic estimates usable on larger structured objects for which direct enumeration is infeasible, while AID can use perturbations of candidate structures to identify components whose modification changes estimated algorithmic organisation or behaviour. BDM and AID do not themselves turn the bounded predictor into the exact Solomonoff semimeasure, nor do they guarantee monotonic improvement in predictive accuracy. Their role is to make the finite algorithmic search more scalable and intervention-oriented. In that qualified sense, an open-ended CTM process extended by BDM and AID provides a route towards progressively richer, resource-bounded Solomonoff-style induction rather than a claim to compute Solomonoff induction exactly.

The difference in the use of computation can also be written directly. Let c index a computational resource budget and, for an observed prefix x , define the accessible generating programs

$$\mathcal{G}_c(x) = \{p : |p| \leq L(c), U(p) \text{ outputs a string with prefix } x \text{ within } T(c) \text{ steps}\}, \quad (27)$$

where $L(c)$ and $T(c)$ are non-decreasing program-length and runtime limits. The set $\mathcal{G}_c(x)$ is thus the program hypothesis space that can actually be searched at budget c . Since $c_1 < c_2$ implies $\mathcal{G}_{c_1}(x) \subseteq \mathcal{G}_{c_2}(x)$, additional computation can make new generative mechanisms available rather than merely optimise parameters within a fixed hypothesis class. The bounded prefix mass in (25) is precisely a weighted sum over such an accessible set. Increasing computation in this construction can make a previously excluded generative mechanism enter the hypothesis space. This is categorically different from merely reducing the optimisation error ε_c in (22) while the likelihood target and hypothesis class remain fixed.

Seen in this way, CTM changes the role assigned to additional computational power. In likelihood-based training, more computation devoted to the same objective principally

improves optimisation or supports a larger chosen model, but does not by itself supply the universal program prior. In an open-ended CTM regime, increased computation can enlarge the program classes and runtimes from which algorithmic probability is estimated. BDM provides a route for applying related estimates to structures beyond the scale of direct enumeration, while AID provides a perturbational framework for examining how candidate structures respond to modification. The resulting system therefore places progressively more of the search for explanatory structure inside the computational process itself. This does not remove dependence on the reference machine, representation or empirical evidence, but it gives increased computation a qualitatively different inferential role.

The elementary even-number example is intentionally simple, but the same distinction appears whenever observations underdetermine a mechanism. Consider two dynamical systems that agree over the observed time window but diverge after a perturbation. A flexible predictor may fit both histories equally well. A mechanism-oriented learner instead attempts to construct candidate update rules and can ask which intervention separates them. The advantage is not guaranteed correctness. It is that the hypothesis is expressed in a form from which discriminating consequences can be derived.

A second example is scientific extrapolation outside observed support. Suppose a small collection of measurements is consistent with both a low-degree law and a high-capacity interpolating function. Within the measured range their predictive errors may be indistinguishable. The compact law makes a stronger commitment about what should happen elsewhere. That commitment is risky, but precisely because it is risky it can be tested. Mechanism synthesis converts fit into a source of counterfactual or extrapolative questions. A distributional model can of course learn the same law internally; the point is functional, not architectural. What matters is whether the system can expose and manipulate the structure sufficiently to derive and test such commitments.

This is the sense in which model synthesis is closer to scientific inference than unrestricted generation. A scientific model earns epistemic value not because it produces novel statements, but because it compresses observations into a structure that entails further consequences and survives attempts to falsify them. The structure may be an equation, a causal graph, a program, a simulator or another executable representation. Symbolic model synthesis is used in this paper as a collective name for that function, not as a claim that classical symbolic AI is the only implementation capable of it.

A neurosymbolic architecture is useful here if the term is understood functionally. Statistical models are powerful proposal generators and estimators. Symbolic objects such as program s, equations, causal graphs, type constraints, conservation laws and proof objects provide forms that can be executed, composed, perturbed or checked. An effective system need not choose between them. A neural component can propose a candidate mechanism, an algorithmic or symbolic component can make its structure explicit, and an external observation, intervention or verifier can decide whether the consequences survive testing.

The central benefit is not that symbolic structure magically restores lost data or guarantees a contraction toward truth. Structural priors can be wrong. A short program may fit the observations for the wrong reason. A causal graph may be unidentifiable from the available interventions. CTM estimates depend on the finite machine class and runtime cutoff, BDM on decomposition, and AID on the perturbation model and interpretation. Model synthesis therefore needs independent validation rather than replacing it.

Four practical failure modes are worth distinguishing. There is a representation failure when the chosen language contains no concise adequate mechanism; a budget failure when an adequate mechanism exists but lies outside the current length or runtime limits; an identification failure when several accessible mechanisms remain compatible with the evidence; and a search or verification failure when an accessible mechanism is not found or is tested inadequately. These are operationally different problems. Calling them all “lack of information” hides precisely

the methodological distinctions that matter in empirical discovery.

The functional loop proposed here is consequently simple to state. Generate candidate mechanisms, derive consequences, test those consequences against evidence or invariant rules, and retain the validated changes. CTM, BDM and AID are possible ingredients in the mechanism-proposal and structural-analysis stages, not exceptions to information conservation. Their role is to enlarge or reorganise access to mechanism space. The direction of improvement still comes from successful contact with a criterion that is not merely an echo of the current model distribution.

There is an older epistemological distinction behind this argument. Novel expression is not the same as new warranted knowledge. Generative models make this unusually visible because they can produce strings, images, proofs and hypotheses that have never previously occurred while remaining entirely within a fixed computational interaction with their inputs and parameters. Novelty in the output space is therefore cheap evidence for epistemic progress. The harder question is whether the generated object establishes a relation to the world, a formal system or an engineering constraint that was not previously warranted.

The Kantian analytic/synthetic distinction offers an analogy, although not a machine-learning theorem. An inference can unpack consequences of what is already represented without thereby acquiring a new empirical fact. Scientific progress typically combines such internal derivation with observations that can defeat the current theory. Algorithmic search occupies the first side of this cycle when it exposes consequences implicit in data and priors; experiments and interventions occupy the second when they alter the evidential state. The strength of a self-improving architecture lies in coupling them.

This perspective also helps distinguish explanation from prediction. Predictive adequacy is indispensable, but a mechanism has additional uses when it supports intervention, counterfactual reasoning, composition with other mechanisms or transfer to regimes not represented in the training distribution. None of these properties follows merely from assigning high probability to observed data. Conversely, an explicit mechanism that predicts badly is not rescued by being interpretable or simple. The argument is for a cycle in which generative structure makes stronger commitments and empirical or formal evaluation decides whether those commitments deserve to survive.

7 Finite empirical tests of resource-bounded mechanistic search

7.1 Empirical protocol

For illustrative purposes, we tested the finite operational predictions of Eqs. (24)–(27) in an explicit prefix-free program space rather than attempting to approximate exact Solomonoff induction. The benchmark contained 11,250 unique deterministic binary mechanisms generated by four families—periodic rules, affine modular threshold rules, XOR-lag recurrences, and Walsh-like generators. Forty-eight ambiguous target mechanisms were selected, balanced equally across the four families (12 per family). Each target generated a 128-bit sequence. For the fixed-observation experiment, only the first 16 bits were observed and held unchanged while the accessible description-length budget was increased from $L = 4$ to 26 bits; prediction was evaluated on the following 64 bits.

For each accessible budget, the finite program predictor assigned a mechanism the complete bounded mass

$$m_L(x) = \sum_{\substack{p: U(p)=x \\ |p| \leq L}} 2^{-|p|},$$

summing over all program descriptions in the finite language rather than using only the shortest description. We compared this predictor with a uniform mixture over prefix-compatible

mechanisms, a likelihood-only order-8 Krichevsky–Trofimov autoregressor, a depth-8 Context-Tree Weighting (CTW-D8) predictor prespecified to match the order-8 statistical baseline, and a small task-local Transformer trained solely by next-token cross-entropy. The Transformer used two encoder layers, four attention heads, embedding dimension 48, AdamW ($\text{lr} = 3 \times 10^{-3}$, weight decay 10^{-4}), and 450 optimisation steps on the same observed prefix. It is intended as a controlled likelihood-based neural baseline, not as a proxy for a frontier LLM.

Two additional structural priors were computed independently over the same candidate mechanisms. BDM was evaluated with `pybdm` implementation in one dimension. A second code-length prior used the bundled `pycdm` implementation and its symmetrised certified code-length. These quantities are finite complexity estimators. CTW-D8 was included as a stronger Bayesian/universal-coding statistical baseline: it performs context-tree mixture prediction without explicit search over the finite mechanism class used by the program-space predictors.

A second experiment compared three recursive update loops from the same initial eight observed bits over 14 update cycles. In closed self-training, each additional accepted bit was generated by the currently selected mechanism itself. In the random-verifier condition, a randomly selected unresolved position was provided by the true external mechanism. In the active program-plus-verifier condition, the unresolved position of maximal posterior Bernoulli entropy among the highest-mass candidate mechanisms was queried and its true externally supplied value was added to the evidence. Outcomes were external 64-bit continuation accuracy, consistency with accepted evidence, exact mechanism recovery, and restricted mean cycles to recovery.

All primary comparisons were paired by target mechanism. Means and differences are reported with 4,000-resample bootstrap 95% confidence intervals. Paired comparisons use 10,000 sign-permutation draws with Benjamini–Hochberg false-discovery-rate correction, and exact-recovery proportions use Wilson intervals.

7.2 Resource expansion exposes predictive mechanisms under fixed evidence

Holding the observed prefix fixed while enlarging the accessible mechanism class produced a clear gain in extrapolative performance (Fig. 2a). At the maximum program budget, the cross-entropy Transformer achieved mean continuation accuracy 0.544 (95% CI 0.512–0.579). The uniform mechanism mixture reached 0.748 (0.717–0.780; paired $\Delta = 0.204$, FDR $q = 5.0 \times 10^{-4}$), the pyCDM prior reached 0.717 (0.644–0.792; $\Delta = 0.173$, $q = 5.0 \times 10^{-4}$), the finite program-mass predictor reached 0.654 (0.600–0.705; $\Delta = 0.110$, $q = 8.3 \times 10^{-4}$), and the pyBDM prior reached 0.643 (0.574–0.714; $\Delta = 0.099$, $q = 0.0215$). The likelihood-only KT baseline reached 0.571 and did not significantly differ from the Transformer after correction ($q = 0.4168$). CTW-D8 likewise showed no extrapolation advantage, reaching 0.538 (95% CI 0.493–0.585; $\Delta = -0.006$, FDR $q = 0.7523$), with exact mechanism recovery of only 2.1%. As Fig. 2b shows, CTW nonetheless achieved substantially lower next-bit log loss (0.814 [0.601, 1.041]) than the Transformer (6.890 [4.230, 9.635]) without improving out-of-support extrapolation, separating local predictive coding from explicit mechanism-space inference as argued herein.

Exact mechanistic recovery further confirmed the same distinction (Fig. 2c): the Transformer recovered none of the 48 target mechanisms, whereas pyCDM recovered 37.5% (95% Wilson CI 25.2–51.6%), pyBDM 27.1%, the uniform mechanism mixture 16.7%, and finite program mass 10.4%. The result supports the resource-accessibility claim rather than a universal ordering of complexity priors: the uniform candidate mixture gave the strongest average extrapolation and next-bit calibration, whereas pyCDM gave the highest exact mechanism recovery. What consistently separated the successful methods from the fixed likelihood baseline was access to an explicit compatible mechanism class whose contents changed with the search budget.

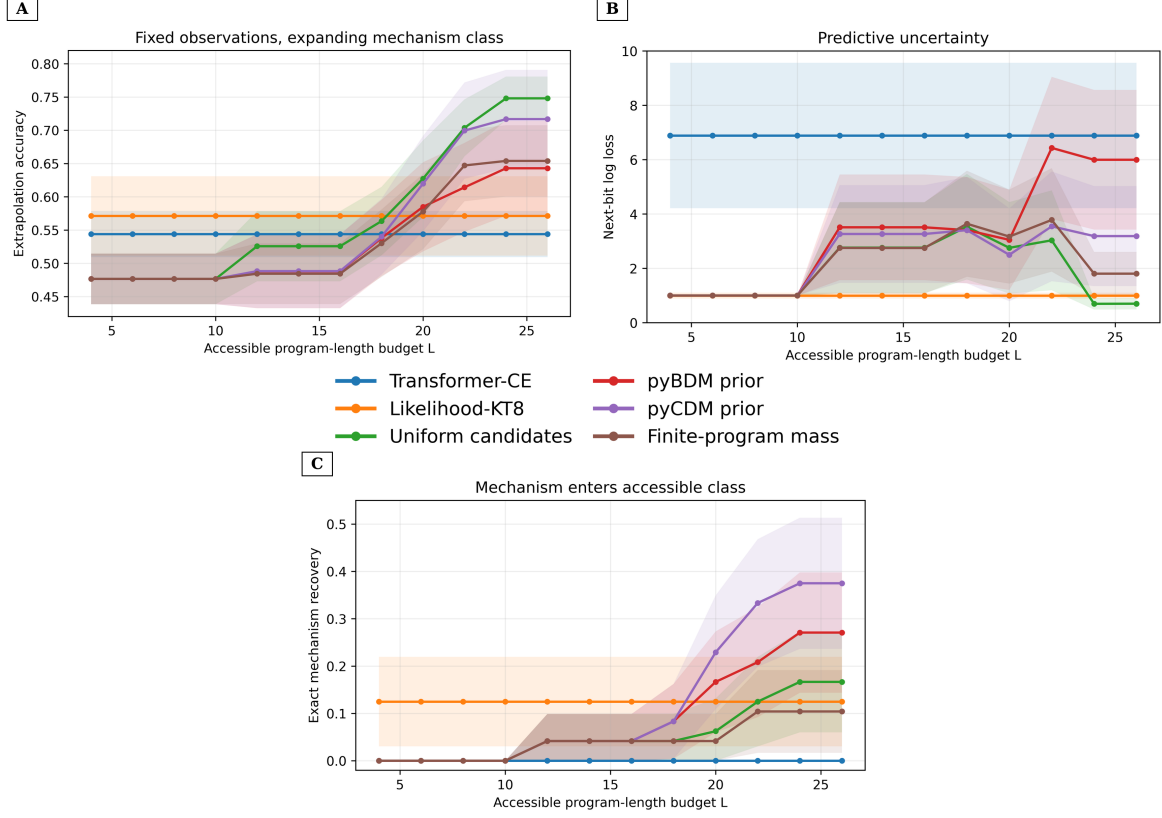


Figure 2: **Fixed observations with an expanding accessible mechanism class.** Forty-eight ambiguous short-program environments were evaluated while the first 16 observations remained unchanged and the maximum admissible program length L increased. **(A) Held-out 64-bit extrapolation accuracy.** With the observed prefix fixed, increasing the accessible program-length budget L improves extrapolative accuracy only for predictors that search an explicit compatible mechanism class. Likelihood-only baselines remain flat. Shading denotes 95% bootstrap confidence intervals across target mechanisms. **(B) Next-bit log loss.** Access to additional compatible mechanisms changes predictive uncertainty for the mechanism-space methods, whereas the fixed likelihood baselines do not improve with larger search budgets. Lower values indicate better local predictive coding. Shading denotes 95% bootstrap confidence intervals across target mechanisms. **(C) Exact recovery of the generating continuation.** Enlarging the accessible mechanism class increases the fraction of targets for which the correct generator enters the supported search space and is recovered exactly, again in contrast to likelihood-only baselines. Shading denotes 95% bootstrap confidence intervals across target mechanisms. The three panels separate extrapolative accuracy, local predictive coding, and exact mechanism recovery. The experiment tests finite resource-bounded accessibility and does not claim computation of exact Solomonoff induction.

Table 1: **Fixed-observation prediction at the maximum accessible program budget.** Confidence intervals are bootstrap 95% intervals for accuracy and log loss and Wilson intervals for exact recovery. q values are Benjamini–Hochberg-adjusted paired comparisons with the Transformer. CTW-D8 provides a stronger statistical/universal-coding baseline: despite competitive next-bit coding, it does not reproduce the extrapolation or mechanism-recovery gains of explicit mechanism-space search. Across the mechanism-space methods, uniform candidate averaging gives the strongest mean extrapolation, whereas pyCDM gives the highest exact mechanism recovery.

Method	Extrapolation accuracy	Δ accuracy vs. Transformer	Next-bit log loss	Exact recovery	FDR q
Transformer-CE	0.544 [0.511, 0.579]	–	6.890 [4.230, 9.635]	0.000 [0.000, 0.074]	–
Likelihood-KT8	0.571 [0.514, 0.632]	0.027 [−0.025, 0.085]	0.998 [0.895, 1.104]	0.125 [0.059, 0.247]	0.4168
CTW-D8	0.538 [0.493, 0.585]	−0.006 [−0.038, 0.028]	0.814 [0.601, 1.041]	0.021 [0.004, 0.109]	0.7523
Uniform candidates	0.748 [0.716, 0.781]	0.204 [0.162, 0.248]	0.700 [0.508, 0.917]	0.167 [0.087, 0.296]	0.0006
pyBDM prior	0.643 [0.573, 0.710]	0.099 [0.024, 0.177]	5.997 [3.505, 8.605]	0.271 [0.166, 0.410]	0.0266
pyCDM prior	0.717 [0.645, 0.788]	0.173 [0.095, 0.249]	3.187 [1.474, 5.105]	0.375 [0.252, 0.516]	0.0008
Finite-program mass	0.654 [0.600, 0.706]	0.110 [0.054, 0.167]	1.805 [1.044, 2.619]	0.104 [0.045, 0.222]	0.0008

7.3 Independent verification converts recursion into cumulative improvement

The recursive experiment directly separated internal consistency from externally verified improvement. Figure 3a shows that closed self-training remained nearly flat in external performance, while Fig. 3b shows that it nevertheless remained perfectly consistent with the evidence it had accepted. Its final external continuation accuracy remained 0.538 (95% CI 0.494–0.587), essentially its initial level, and exact mechanism recovery in Fig. 3c was only 2.1%. Thus recursive agreement with self-generated evidence was not itself an improvement signal.

Replacing endogenous evidence by independent observations changed the dynamics. Random external verification produced final external accuracy 0.990 (0.979–0.998), exact recovery 87.5%, and a mean restricted time to recovery of 7.25 cycles. Active mechanism-guided querying followed by the same independent verifier reached external accuracy 1.000, exact recovery 100%, and restricted mean time to recovery of 5.04 cycles. Relative to closed self-training, the external-accuracy gains were 0.452 and 0.462, respectively, with FDR-adjusted $q = 9.999 \times 10^{-5}$ for both comparisons. Crucially, Fig. 3b shows that consistency with the accepted evidence was 1.0 in all three conditions throughout, whereas Fig. 3c shows that only the independently verified loops progressively recovered the correct generating mechanism. Internal agreement therefore failed to distinguish the stationary closed loop from the two externally improving loops, whereas independent verification did.

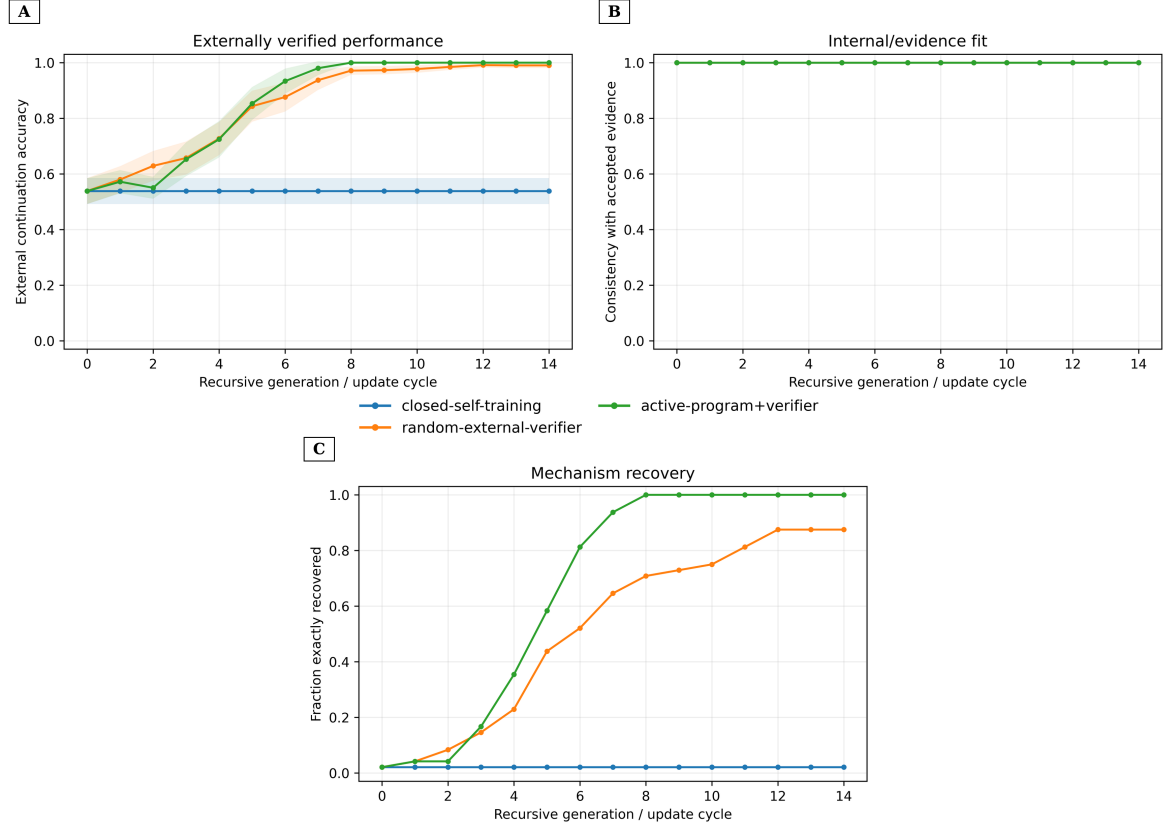


Figure 3: **Recursive self-training versus independently verified mechanism search.** **(A) Externally evaluated continuation accuracy.** Closed self-training stays nearly flat, while independently verified recursion rapidly improves external performance. Random verification helps, and active program-guided querying helps most. Shading denotes 95% bootstrap confidence intervals across target mechanisms. **(B) Consistency with accepted evidence.** All three loops remain perfectly consistent with the evidence they accept, showing that internal agreement alone does not diagnose actual improvement. **(C) Exact recovery of the generating mechanism.** Independent verification progressively restores the true mechanism, and active program-guided querying achieves complete recovery earlier than random verification. The three panels disentangle external task improvement, internal evidence fit, and exact recovery of the hidden generator over recursive update cycles. Closed self-training remains internally coherent without improving externally, whereas independently verified recursion restores the true mechanism.

Table 2: **Recursive-loop outcomes after 14 update cycles.** Restricted mean cycles count unrecovered targets at the fixed 15-cycle censoring boundary. Accuracy comparisons are paired against closed self-training and FDR corrected.

Loop	Final external accuracy	Exact recovery	Restricted mean cycles to recovery	Δ Accuracy vs. closed FDR q
Closed self-training	0.538 [0.494, 0.587]	0.021 [0.004, 0.109]	14.69 [14.06, 15.00]	—
Random external verifier	0.990 [0.979, 0.998]	0.875 [0.753, 0.941]	7.25 [6.15, 8.44]	0.452 [0.399, 0.500] $q = 9.999 \times 10^{-5}$
Active program + verifier	1.000 [1.000, 1.000]	1.000 [0.926, 1.000]	5.04 [4.56, 5.50]	0.462 [0.414, 0.505] $q = 9.999 \times 10^{-5}$

8 Consequences for AGI, ASI and the so-called Singularity

The results in this paper isolate a missing step in a common narrative. The ability to generate one’s own future training data does not establish an improvement operator. Exact self-training is stationary; finite self-replacement may drift or lose diversity; and a corrective signal changes the process precisely because it introduces a criterion not contained in neutral self-imitation.

Good’s intelligence explosion requires a process in which improvements to the system increase its capacity to produce further improvements [1]. Vinge and Kurzweil place related feedback at the centre of their accounts of technological discontinuity [2, 3]. For contemporary generative AI, the question is therefore concrete: which part of the system proposes a change, which part decides that the change is genuinely better, what evidence supports that judgement, and how does the system prevent the evaluator from becoming circular as the generator changes?

A closed recursive language-modelling loop has an incomplete answer. It can produce vast numbers of candidates, including code, plans and critiques. It can also imitate evaluation language. But if candidate generation and evaluation are ultimately trained to agree with the same endogenous distribution, agreement is not an independent measure of truth, causal adequacy or general capability. More generations of the same loop do not solve that epistemic problem simply by increasing scale.

Externally anchored systems are different. A model that writes faster code and runs it against a trusted performance benchmark has a real improvement signal. A model that proposes a proof and submits it to a sound proof checker has a correctness criterion independent of its confidence. A model that designs an experiment and receives a measurement from the world changes its information state. A model that plays a formal game can improve through self-play because the rules and outcome remain invariant even when all trajectories are synthetic. None of these systems is closed in the relevant sense.

The distinction also prevents a misleading equation between autonomy and collapse. A future system could be highly autonomous while continuously seeking external evidence. Indeed, greater capability may allow it to design better experiments, build better instruments or find stronger formal tests. Such a system would not resemble a passive stream of fresh human-authored text, yet it would remain epistemically open. The concern addressed here applies when the feedback loop becomes progressively self-referential, not when human supervision merely decreases.

This is where symbolic model synthesis enters the picture. An intelligence explosion, if it is to involve open-ended scientific and technological competence rather than only increasingly efficient distribution fitting, needs a way to represent candidate mechanisms in forms that can be modified and tested. program synthesis is one route. Causal models, equations, formal proofs and executable simulators are others. The requirement is functional: the system must be able to move from a predictive surface to candidate generative structure and back to independent tests.

Nothing in this argument proves that only CTM or AID can provide that capability. Nor does it establish that a transformer cannot internalise the relevant operations. If a neural system reliably constructs executable mechanisms, derives counterfactual or extrapolative consequences and tests them against independent evidence, then it has implemented the required model-synthesis layer, whatever its internal architecture is called.

The next-token formulation sharpens this architectural distinction. Two systems can both return a probability distribution over the next token while implementing very different inductive procedures. A conventional LLM obtains that distribution from learned logits optimised primarily for likelihood on observed token sequences. A CTM-augmented system can additionally score continuations or latent mechanisms by finite algorithmic probability, so that explanatory programs compete according to generative description rather than token fit alone. Increasing the capacity or training compute of the former can improve the learned conditional distribution without changing the defining objective. Increasing the search budget of the latter can enlarge the program class over which algorithmic hypotheses are considered. If those hypotheses are then exposed to independent empirical or formal tests and successful mechanisms are retained, increased computational capability can improve the mechanism-search process itself. That is a plausible source of the second-order positive feedback required by recursive self-improvement, although it remains an empirical architectural hypothesis rather than a consequence of algorithmic probability alone.

The paper rejects the inference that recursive LLM generation and retraining, by themselves, already instantiate the positive-feedback mechanism assumed in classical Singularity narratives. A future system may add the missing ingredients. The claim is that those ingredients are substantive and cannot be inferred from recursive density matching alone.

The connection to Good can now be stated more sharply. An ultraintelligent machine must do more than search a fixed design space faster. If its own improvement changes the representation of the problem, the hypotheses it can formulate and the evaluators it uses, then the feedback loop must preserve a notion of success through those changes. Otherwise a system can appear to improve according to an evaluator that it has itself made easier to satisfy. The classical intelligence-explosion premise therefore contains an often implicit epistemic assumption: the process that recognises improvement remains sufficiently anchored while the system responsible for proposing improvements changes.

Vinge’s argument raises the same issue from the perspective of predictability. A transition can become difficult for humans to forecast because the system explores designs or theories outside our effective search horizon. That does not remove the need for the system itself to discriminate successful from unsuccessful changes. Indeed, the farther its search moves beyond human supervision, the more the architecture depends on reliable non-circular tests. Autonomy shifts the burden of validation from people to the system’s interaction with formal and empirical constraints.

Kurzweil’s account emphasises accelerating returns across interacting technologies. Such acceleration can provide more computation, more data and better tools, all of which can enlarge the accessible hypothesis space. The argument here does not deny those trends. It identifies a separate requirement. More resources can accelerate search, but search becomes recursive self-improvement only when candidate changes are connected to a criterion that tracks genuine capability and when the resulting improvements feed back into the capacity to discover and

validate further changes.

We argue that that the familiar inference from increasingly autonomous LLM generation to a near-term Singularity omits an architectural step. Present systems are extraordinarily capable proposal generators and increasingly capable tool users. The unresolved question is whether they possess, or are being given, a sufficiently general mechanism-synthesis and validation loop to turn recursive generation into reliable recursive improvement across changing domains.

This formulation makes the thesis falsifiable. A strong counterexample would be a genuinely distribution-only recursive system that, without functionally equivalent mechanism construction or independent evaluation, exhibits sustained externally verified discovery and improvement across changing domains. Evidence in the opposite direction would come from systems in which explicit mechanism search plus independent verification produces improvements that self-training alone cannot sustain under matched data and computational budgets.

The thesis is deliberately implementation-neutral even if we have introduced CTM, BDM and AID methods as tools closer to AIT more capable of Solomonoff induction than other more statistical tools.

A transformer that internally learns to construct executable models, calls tools to test them, revises them after failed predictions and preserves the resulting improvements would satisfy the requirement. Conversely, attaching a symbolic module that merely reformats the model's own assertions without independent evaluation would not.

This functional reading matters because neural and symbolic computation are not mutually exclusive categories at the level relevant to the argument. Neural systems can implement algorithms; symbolic systems can use statistical search; hybrid systems can move between continuous representations and discrete executable objects. The architectural distinction of interest is between a loop whose criterion is endogenous distributional agreement and a loop that can construct commitments and expose them to tests capable of rejecting the current model.

A useful way to view the scientific method in this context is as repeated alternation between compression and resistance. A theory compresses observations by replacing many facts with a smaller generative account. From that account one derives consequences. The world then resists by failing to behave as predicted, or a formal system resists by rejecting a proof or violating a constraint. The resulting discrepancy drives revision. A self-improving system does not need to imitate human scientific institutions, but an open-ended system needs some mechanism with the same logical role: proposals must encounter something they do not control.

This also explains why simply adding more real data is not the deepest solution to recursive closure. Passive data accumulation can stabilise estimation and broaden coverage, but scientific progress often depends on choosing which data to obtain. An experiment is valuable because competing models predict different outcomes. A capable agent can therefore increase its grounding not only by consuming an external stream but by actively creating informative interactions with the world. In that regime the learner's intelligence can itself improve the quality of the evidence it obtains, providing a plausible positive feedback that is absent from neutral self-training.

The converse risk is evaluator endogeneity. As more of the training and evaluation stack is generated by the system, benchmark success can cease to track the intended capability. Reward hacking is one familiar form, but the issue is broader: a system may narrow its ontology, choose experiments that cannot falsify its favoured models, or optimise formal specifications that omit the property of interest. The requirement of non-circular evaluation should therefore be understood dynamically. Evaluators, instruments and formal specifications may themselves be revised, but their revision must remain exposed to constraints that are not simultaneously rewritten to guarantee success.

These considerations suggest a research program beyond model-collapse measurements. One line of experiments should compare systems under the same fixed observations while varying

the computational and representational resources available for mechanism search. Another should allow systems to select interventions and measure how efficiently they distinguish observationally equivalent hypotheses. A third should compare recursive self-training with recursive model synthesis under the same total compute, including deliberately misaligned priors and imperfect verifiers. The important outcome is not whether one method produces more novel text, but whether it generates hypotheses whose independently tested consequences improve over successive cycles.

A particularly strong test would place a learner in a family of environments whose surface statistics change while their generating mechanisms share reusable structure. Pure distribution matching should have to relearn much of the surface variation. A successful mechanism-synthesis system should increasingly exploit the reusable generative structure, propose discriminating interventions and transfer validated components to new environments. If a distribution-only recursive system achieved the same sustained, externally verified improvement without any functionally equivalent mechanism construction or independent evaluation, it would count directly against the thesis advanced here.

9 Discussion and limitations

The paper began from model collapse, but its main conclusion is about the structure of improvement. Collapse is one possible outcome of finite recursive replacement. Stationarity is the exact population outcome of pure self-imitation. Neither is an intelligence explosion. Once external data, replay, correction, tools, reward models, formal constraints or environmental feedback enter, the effective update operator changes and must be analysed on its own terms. This is why the empirical synthetic-data literature should not be compressed into a single universal recurrence.

The mathematics is intentionally simple. The exact diversity result uses multinomial sampling on a finite alphabet, and the grounding calculation assumes a fixed reference distribution. Real neural training adds misspecification, optimisation state, regularisation, retrieval, architecture changes, non-stationary objectives and selective data curation. These can alter the dynamics substantially. The equations isolate mechanisms rather than simulate a frontier training pipeline. They support claims about the specified loops, not universal claims about every neural system.

The same qualification applies to the comparison between likelihood-based learning and Solomonoff induction. Cross-entropy is not intrinsically tied to a particular neural architecture, and the argument does not claim that a likelihood-trained model cannot learn program-like internal structure. Nor does it claim that all useful distribution curation must be performed by humans. A system may learn to choose its own data, curricula or interventions. When it does so, those learned operations are additional components of the effective estimator and should be evaluated as such. The narrower claim is that the likelihood objective by itself neither chooses the curation function in (17) nor supplies the program weighting characteristic of Solomonoff induction. Likewise, C_{mass} in (21) measures distributional displacement only; it is not a measure of intelligence, labour or epistemic value.

The resource-bounded accessibility formulation is likewise an abstraction. To turn it into a complete theory one would need explicit complexity classes, task losses, runtime distributions and representation constraints. Its purpose here is narrower: to make clear why the truth of DPI or Levin non-growth does not settle an empirical comparison between finite procedures. Two methods can obey the same global bound while producing very different predictions because one can operationally reach a useful mechanism and the other cannot.

Our CTW control reinforces this distinction: substantially improved next-bit universal coding did not translate into improved extrapolation or mechanism recovery, indicating that

predictive compression of the observed prefix and explicit access to compatible generative mechanisms are operationally distinct.

The Kantian language should also be interpreted carefully. Kant’s analytic and synthetic distinction belongs to a philosophical framework that is not a theorem about machine learning [27]. The useful analogy is epistemological: producing a novel string is different from establishing a new, warranted claim about the world. A system may generate highly original hypotheses from existing information, but the epistemic status of those hypotheses depends on how they are tested.

Causality introduces its own qualifications. Observational data can sometimes identify causal structure under strong assumptions, and interventions are not the only possible source of causal information. Conversely, interventions do not automatically solve every identification problem. The paper relies only on the weaker point that observational distribution matching does not by itself select a unique causal mechanism [28]. Any causal or algorithmic model proposed by a self-improving system must still confront identifiability, misspecification and verification.

Algorithmic methods carry parallel limitations. Kolmogorov complexity is uncomputable, CTM is a finite approximation, BDM depends on decomposition choices, and AID perturbation scores require a specified representation and perturbation scheme [25, 26, 29]. The invariance theorem controls machine dependence only up to an additive constant, which can be important for short objects. Simplicity is a prior, not a guarantee of truth. These limitations are reasons to couple structural inference to evidence rather than reasons to abandon it.

The independent evaluator in a recursive self-improvement loop can itself fail. Benchmarks can be gamed, proof systems can formalise the wrong specification, experiments can be poorly designed, and reward models can become endogenous proxies. The architectural solution requirement is therefore not merely to add a verifier but to maintain a chain of correction in which proposed improvements remain answerable to criteria whose failure modes are themselves exposed to independent testing. As the system improves, preserving that non-circularity may be harder than generating candidate changes.

The finite experiments in Section 7 provide some support for these distinctions. With the observations fixed, expanding an explicit mechanism class improved extrapolation and mechanism recovery relative to the task-local likelihood baseline. For instance, in the recursive experiment, perfect consistency with self-generated evidence coexisted with stationary external performance, whereas independently verified observations produced cumulative improvement and mechanism recovery. Further, the active program-guided verifier reached recovery in fewer cycles than random external querying. These results support our architectural claim that mechanism representation and non-circular evaluation perform distinct work that cannot be solely inferred from recursive distributional agreement. They do not establish that any particular algorithmic prior is universally optimal: the uniform mechanism mixture achieved the strongest mean extrapolation in the fixed-prefix benchmark, and the pyBDM/pyCDM robustness analyses confirmed representation dependence. Our evidence supports the mechanism-search and epistemic-openness thesis more strongly than any claim of universal superiority of a single complexity estimator.

The methodological thesis was tested directly by holding observations fixed while expanding the computationally accessible mechanism class. Across 48 ambiguous short-program environments, the first 16 observations remained unchanged while the admissible program-length budget increased, allowing additional compatible generators to enter the finite search space. At the maximum budget, the likelihood-trained Transformer achieved extrapolation accuracy of 0.544 [0.511, 0.579], compared with 0.748 [0.716, 0.781] for the uniform mechanism mixture, 0.717 [0.645, 0.788] for pyCDM, 0.654 [0.600, 0.706] for finite-program mass, and 0.643 [0.573, 0.710] for pyBDM (Fig. 2; Table 1). To recall, the uniform mechanism mixture assigns equal prior weight to every candidate generating program consistent with the observed prefix, then

averages their predictions without favouring shorter programs. Exact mechanism recovery was 0% for the Transformer, compared with 37.5% for pyCDM, 27.1% for pyBDM, 16.7% for the uniform mixture, and 10.4% for finite-program mass. These results highlight the distinction in (14): enlarging the accessible hypothesis class can make consequences of fixed evidence computationally available without violating the information bound in (12).

The next-token control revealed a dissociation between predictive coding and mechanism-level extrapolation. CTW-D8 achieved lower next-bit log loss than the Transformer, 0.814 [0.601, 1.041] versus 6.890 [4.230, 9.635], yet showed no corresponding extrapolation advantage: accuracy was 0.538 [0.493, 0.585] versus 0.544 for the Transformer (paired $\Delta = -0.006$, FDR $q = 0.7523$), and exact mechanism recovery remained only 2.1%. The likelihood-only KT8 baseline similarly showed no significant extrapolation advantage ($q = 0.4168$). Thus, substantially better next-bit coding was not accompanied by better out-of-support extrapolation or mechanism identification. By contrast, explicit mechanism space predictors improved extrapolation as compatible generators became accessible. The uniform mixture achieved the highest mean extrapolation accuracy, whereas pyCDM achieved the highest exact recovery. These results distinguish local statistical prediction from generative mechanism inference in this finite benchmark. They support access to explicit mechanism classes rather than the universal superiority of any particular algorithmic prior, and do not establish that the Transformer is incapable of implementing such inference under other architectures or training regimes.

The recursive experiment independently demonstrated a dissociation between internal consistency and externally verified improvement. All three recursive arms maintained identical perfect consistency with their accepted evidence (1.0) throughout, yet their external outcomes diverged sharply (Fig. 3; Table 2). Closed self-training remained approximately stationary, with final external accuracy of 0.538 [0.494, 0.587] and only 2.1% exact recovery. Random independent verification increased accuracy to 0.990 [0.979, 0.998] and recovery to 87.5%, while active program-guided querying followed by the same verifier reached 1.000 accuracy and 100% recovery. Accuracy gains over closed self-training were 0.452 and 0.462, respectively (FDR-adjusted $q = 9.999 \times 10^{-5}$ for both); active querying also reduced restricted mean recovery time from 7.25 to 5.04 cycles relative to random verification. Crucially, perfect internal agreement could not distinguish the stationary closed loop from the improving verified loops: only independent evidence progressively corrected the inferred mechanism. These finite experiments support the distinct roles of mechanism space accessibility and non-circular verification, without establishing a universal theorem about LLMs or Solomonoff induction. Reference-language sensitivity, deliberately misaligned priors, matched computational-cost comparisons, and broader replacement, accumulation, replay and correction regimes remain necessary tests of generality in prospective studies. Regardless, we can generally conclude that better next-bit coding did not imply better mechanism extrapolation, and perfect internal consistency did not imply recursive improvement.

Finally, this paper does not derive a probability or date for AGI, ASI or the Singularity. It does not claim that symbolic model synthesis is sufficient for any definition of general intelligence, that other neural system architectures cannot perform it, or that every self-training system collapses. It makes a more specific claim about the route from recursive generative modelling to recursive self-improvement. An intelligence explosion requires an improvement operator. Closed-loop density matching does not supply one. Mechanism construction and non-circular evaluation are therefore not decorative additions to the Singularity story; they are part of what the positive-feedback premise still has to explain.

10 Conclusions

Recursive self-training should not be identified with recursive self-improvement. In the exact population limit, self-training under log-loss reproduces the current distribution. Under finite replacement, repeated resampling can reduce diversity and introduce stochastic distortion. External signals can stabilise the process, but their instantaneous proportion is not decisive. Cumulative corrective influence, sample size, retention and the form of the update determine the simple dynamics analysed here.

This correction does not weaken the architectural point. It sharpens it. The relevant danger is not the problem of vanishing real-data percentage, as a universal law but of epistemic closure: a loop in which candidate generation, training targets and evaluation become increasingly endogenous. A highly autonomous system need not be epistemically closed if it continues to interact with the world, use invariant verifiers or preserve other independent constraints.

The information-theoretic clarification is equally important. DPI and Levin non-growth remain fully valid. Their role is foundational: fixed processing cannot create information about an external mechanism from nothing. Their methodological force is limited in the finite empirical regime because they do not tell us which representation, program search or computational budget will expose useful structure already present in partial observations. Resource-bounded accessibility can increase while the global information ceiling remains unchanged.

CTM, BDM and AID should therefore be understood as tools for mechanism search and structural inference where others can exist, but not as exceptions to information conservation and not as substitutes for empirical grounding. Their potential contribution is to organise a search over executable generative explanations and to make structural hypotheses available for perturbation and testing. They can fail, and their outputs require independent validation.

A further conclusion follows for the relation between LLMs and Solomonoff induction. Cross-entropy, negative log-likelihood, maximum likelihood and direct equivalents such as perplexity-based model comparison optimise predictive fit to token distributions; they do not, on their own, supply the defining Solomonoff mixture over generative programs weighted by algorithmic description length. The fact that an LLM chooses its next token from a learned conditional distribution therefore does not make it a Solomonoff predictor. A sufficiently capable neural system may learn a bounded algorithmic procedure internally, but that would be an additional learned capability, not a property guaranteed by the likelihood objective or the softmax next-token interface.

This conclusion is compatible with the constructive result of Wan and Mei when their computable surrogate is distinguished from the likelihood objective itself [18, 19]. Their construction shows how a trained LLM, arithmetic coding, prefix encoding and universal-machine machinery can jointly define a computable program family whose mass is contained within the Solomonoff mixture. That is an important route from an ordinary predictor towards a Solomonoff-style surrogate. The present result identifies where the extra inductive structure enters. Cross-entropy determines the fit of the model to the supplied distribution; the program-based coding construction determines how that fitted model contributes algorithmic mass. There is therefore no contradiction at this level. The present paper refines the claim by separating likelihood optimisation from the additional conditions under which the resulting computable surrogate can be treated as close to the full Solomonoff prior. Only the stronger assertion that likelihood minimisation alone guarantees convergence to the universal semimeasure would go beyond what the cross-entropy identity establishes.

This also identifies where much of the inductive burden resides. Likelihood-based learning can acquire powerful structural biases through architecture, representation, data selection, weighting, curricula and other transformations of the effective training distribution, but those interventions should be counted as part of the learning system rather than attributed to the likelihood-based objective itself. If an external process must continually reshape the probability

mass seen by the learner in order to obtain the desired inference, then part of the intelligence of the resulting system resides in that curation process. Solomonoff induction relocates more of this burden towards computation over generative hypotheses. The contrast is not that one paradigm uses data and the other computation, but that they distribute the inferential burden differently between observations, external curation, architecture and internal mechanism search.

This difference also changes the significance of scaling computational power. More compute applied to a fixed likelihood objective can reduce optimisation error, support larger parameterisations or enable additional procedures, but the objective does not thereby become Solomonoff induction. In an open-ended CTM-based approximation, by contrast, additional computational resources can be used directly to enlarge the program and runtime classes contributing algorithmic mass. BDM and AID can extend this search towards larger structures and their perturbation. The resulting positive-feedback possibility is therefore not simply that a larger predictor fits more data, but that increased capability may enlarge the mechanism-search process that constructs candidate explanations for subsequent independent testing.

The algorithmic route considered here is constructive rather than prohibitive. CTM provides the direct connection to finite approximation of algorithmic probability; an open-ended CTM process can expand the program classes and runtime budgets it explores. BDM can extend the analysis to larger structured objects, and AID can support perturbational analysis and reprogram mability of candidate mechanisms. The resulting architecture should not be identified with exact Solomonoff induction, which remains uncomputable. It is better characterised as a progressively extensible, resource-bounded Solomonoff-style estimator coupled to structural analysis. When its candidate mechanisms are tested against independent evidence or formal constraints, such an architecture supplies a concrete way in which more computational capability can enlarge mechanism search and potentially improve the process that generates further improvements.

The original Singularity question can then be stated more precisely. Good, Vinge and Kurzweil assume a positive feedback in which intelligence makes further intelligence easier to produce. Recursive LLM generation provides an extraordinary engine for producing candidates, but candidate production is only one part of that loop. Sustained self-improvement also requires a way to determine which changes constitute real improvement, to derive consequences that reach beyond self-consistency, and to test them against evidence or invariant criteria.

In that sense, the route from present recursive distributional learning to an intelligence explosion remains incomplete without a functional layer of symbolic model synthesis. That layer need not look like classical symbolic AI and need not be separate from a neural system. What matters is the operation: construct mechanisms, derive consequences, test them non-circularly, and retain what survives. Without that operation, recursion alone is not self-improvement.

Use of AI tools disclosure statement

Generative AI tools were used to assist with language editing, LaTeX formatting, checking derivations in the Sup Mat, proposing objections to arguments, and organising the material. All mathematical arguments, proofs, results, and interpretations originated and were verified by the authors, who assumes full responsibility for the manuscript.

References

- [1] I. J. Good. Speculations concerning the first ultraintelligent machine. *Advances in Computers*, 6:31–88, 1966. doi:10.1016/S0065-2458(08)60418-0.

- [2] V. Vinge. The coming technological Singularity: How to survive in the post-human era. In *Vision-21: Interdisciplinary Science and Engineering in the Era of Cyberspace*, NASA Conference Publication 10129, 1993. NASA Technical Reports Server record 19940022856.
- [3] R. Kurzweil. *The Singularity Is Near: When Humans Transcend Biology*. Viking, 2005.
- [4] S. Alemohammad, J. Casco-Rodriguez, L. Luzi, A. I. Humayun, H. Babaei, D. LeJeune, A. Siahkoochi, and R. G. Baraniuk. Self-consuming generative models go MAD. *International Conference on Learning Representations*, 2024.
- [5] I. Shumailov, Z. Shumaylov, Y. Zhao, N. Papernot, R. Anderson, and Y. Gal. AI models collapse when trained on recursively generated data. *Nature*, 631:755–759, 2024. doi:10.1038/s41586-024-07566-y.
- [6] E. Dohmatob, Y. Feng, P. Yang, F. Charton, and J. Kempe. A tale of tails: Model collapse as a change of scaling laws. In *Proceedings of the 41st International Conference on Machine Learning*, PMLR 235:11165–11197, 2024.
- [7] S. Fu, S. Zhang, Y. Wang, X. Tian, and D. Tao. Towards theoretical understandings of self-consuming generative models. In *Proceedings of the 41st International Conference on Machine Learning*, PMLR 235:14228–14255, 2024.
- [8] M. Gerstgrasser, R. Schaeffer, A. Dey, R. Rafailov, H. Sleight, J. Hughes, T. Korbak, R. Agrawal, D. Pai, A. Gromov, D. A. Roberts, D. Yang, D. L. Donoho, and S. Koyejo. Is model collapse inevitable? Breaking the curse of recursion by accumulating real and synthetic data. arXiv:2404.01413, 2024.
- [9] N. Gillman, M. Freeman, D. Aggarwal, C.-H. Hsu, C. Luo, Y. Tian, and C. Sun. Self-correcting self-consuming loops for generative model training. In *Proceedings of the 41st International Conference on Machine Learning*, PMLR 235:15646–15677, 2024.
- [10] J. Kazdan, R. Schaeffer, A. Dey, M. Gerstgrasser, R. Rafailov, D. L. Donoho, and S. Koyejo. Collapse or thrive: Perils and promises of synthetic data in a self-generating world. In *Proceedings of the 42nd International Conference on Machine Learning*, PMLR 267:29469–29494, 2025.
- [11] A. T. Suresh, A. Thangaraj, and A. N. K. Khandavally. Rate of model collapse in recursive training. In *Proceedings of the 28th International Conference on Artificial Intelligence and Statistics*, PMLR 258:1396–1404, 2025.
- [12] X. Zhu, D. Cheng, H. Li, K. Zhang, E. Hua, X. Lv, N. Ding, Z. Lin, Z. Zheng, and B. Zhou. How to synthesize text data without model collapse? In *Proceedings of the 42nd International Conference on Machine Learning*, PMLR 267:79746–79771, 2025.
- [13] T. M. Cover and J. A. Thomas. *Elements of Information Theory*, 2nd edition. Wiley, 2006. doi:10.1002/047174882X.
- [14] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30*, 2017.
- [15] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever,

- and D. Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems 33*, 2020.
- [16] J. Grau-Moya, T. Genewein, M. Hutter, L. Orseau, G. Deletang, E. Catt, A. Ruoss, L. K. Wenliang, C. Mattern, M. Aitchison, and J. Veness. Learning universal predictors. In *Proceedings of the 41st International Conference on Machine Learning*, PMLR 235:16178–16205, 2024.
 - [17] N. Young and M. Witbrock. Transformers as approximations of Solomonoff induction. arXiv:2408.12065, 2024.
 - [18] J. Wan and L. Mei. Large language models as computable approximations to Solomonoff induction. arXiv:2505.15784, 2025.
 - [19] L. Mei and J. Wan. Large language models as a computable surrogate to Solomonoff induction. ICLR 2026 submission, OpenReview, 2026. <https://openreview.net/forum?id=psDyG1FDzX>.
 - [20] R. J. Solomonoff. A formal theory of inductive inference, Parts I and II. *Information and Control*, 7:1–22 and 224–254, 1964. Part I doi:10.1016/S0019-9958(64)90223-2; Part II doi:10.1016/S0019-9958(64)90131-7.
 - [21] L. A. Levin. Laws of information conservation (non-growth) and aspects of the foundation of probability theory. *Problems of Information Transmission*, 10(3):206–210, 1974.
 - [22] M. Li and P. Vitányi. *An Introduction to Kolmogorov Complexity and Its Applications*, 4th edition. Springer, 2019. doi:10.1007/978-3-030-11298-1.
 - [23] M. Hutter. *Universal Artificial Intelligence: Sequential Decisions Based on Algorithmic Probability*. Springer, 2005. doi:10.1007/b138233.
 - [24] J.-P. Delahaye and H. Zenil. Numerical evaluation of algorithmic complexity for short strings: A glance into the innermost structure of randomness. *Applied Mathematics and Computation*, 219(1):63–77, 2012. doi:10.1016/j.amc.2011.10.006.
 - [25] H. Zenil, S. Hernández-Orozco, N. A. Kiani, F. Soler-Toscano, A. Rueda-Toicen, and J. Tegnér. A decomposition method for global evaluation of Shannon entropy and local estimations of algorithmic complexity. *Entropy*, 20(8):605, 2018. doi:10.3390/e20080605.
 - [26] H. Zenil, N. A. Kiani, F. Marabita, Y. Deng, S. Elias, A. Schmidt, G. Ball, and J. Tegnér. An algorithmic information calculus for causal discovery and reprogram ming systems. *iScience*, 19:1160–1172, 2019. doi:10.1016/j.isci.2019.07.043.
 - [27] I. Kant. *Critique of Pure Reason*. Edited and translated by P. Guyer and A. W. Wood. Cambridge University Press, 1998. doi:10.1017/CBO9780511804649.
 - [28] J. Pearl. *Causality: Models, Reasoning, and Inference*, 2nd edition. Cambridge University Press, 2009. doi:10.1017/CBO9780511803161.
 - [29] H. Zenil, N. A. Kiani, and J. Tegnér. *Algorithmic Information Dynamics: A Computational Approach to Causality with Applications to Living Systems*. Cambridge University Press, 2023. doi:10.1017/9781108596619.

A Supplementary Material

From (6), subtracting P isolates the error relative to the reference distribution:

$$Q_{t+1} - P = (1 - \alpha_t)(Q_t - P).$$

A single grounded update therefore multiplies the discrepancy by $1 - \alpha_t$, so every positive grounding weight contracts it at that step. Repeated substitution gives (7). If some $\alpha_t = 1$, the residual becomes zero immediately. Otherwise the standard infinite-product criterion gives $\prod_t (1 - \alpha_t) = 0$ exactly when $\sum_t -\log(1 - \alpha_t) = \infty$. Since $-\log(1 - x) \geq x$ for $0 \leq x < 1$, divergence of $\sum_t \alpha_t$ is sufficient. If $\sum_t \alpha_t < \infty$, then eventually α_t is small and $-\log(1 - \alpha_t) \leq 2\alpha_t$, so the logarithmic series converges and the product remains positive. Passing to logarithms is what converts the cumulative product into a sum and separates schedules whose many small corrections accumulate without bound from those whose total corrective influence remains finite.

Let $X \sim \text{Multinomial}(N, q)$ and $q' = X/N$. For each coordinate,

$$\mathbb{E}[(q'_i)^2] = q_i^2 + \frac{q_i(1 - q_i)}{N}.$$

The first term is the squared mean of the empirical frequency q'_i , while $q_i(1 - q_i)/N$ is its sampling variance. Their sum gives the second moment needed to track diversity under resampling. Summing over i and using $\sum_i q_i = 1$ yields

$$\mathbb{E}[V(q') \mid q] = \left(1 - \frac{1}{N}\right) V(q).$$

Thus finite sampling reduces expected Gini–Simpson diversity by the factor $1 - 1/N$ in one replacement step. Iteration gives

$$\mathbb{E}V(q_t) = V(q_0) \prod_{s < t} \left(1 - \frac{1}{N_s}\right).$$

The product records the cumulative diversity retained after applying the same one-step contraction through all generations up to time t . Each coordinate is also a bounded martingale because $\mathbb{E}[q_{t+1,i} \mid q_t] = q_{t,i}$. Hence the coordinates converge almost surely. If $\sum_t 1/N_t = \infty$, the product above tends to zero, and bounded convergence implies that the limiting diversity is zero almost surely. On the finite simplex this means convergence to a vertex. The limiting vertex is random rather than selected by an improvement criterion. Equivalently, resampling introduces fluctuations without systematically pushing any coordinate upwards or downwards; convergence to a vertex means eventual fixation on a state in which one outcome carries all probability mass, not directed improvement.

For grounded multinomial replacement, let

$$r_t = \alpha_t p + (1 - \alpha_t) q_t, \quad q_{t+1} = X_t / N_t, \quad X_t \sim \text{Multinomial}(N_t, r_t).$$

Thus r_t is the mixture actually sampled at step t , and q_{t+1} is the empirical frequency vector obtained from N_t multinomial draws from it. Writing $e_t = q_t - p$ gives

$$\mathbb{E}[e_{t+1} \mid q_t] = (1 - \alpha_t) e_t,$$

so the conditional mean error relative to the reference is contracted by the factor $1 - \alpha_t$. The corresponding squared-error recursion is

$$\mathbb{E}[\|e_{t+1}\|_2^2 \mid q_t] = (1 - \alpha_t)^2 \|e_t\|_2^2 + \frac{1 - \|r_t\|_2^2}{N_t}.$$

It separates deterministic contraction from finite-sampling noise: the first term is the squared error remaining after grounding, while the second is the new variance introduced by estimating q_{t+1} from only N_t samples. Let $x_t = \mathbb{E}\|e_t\|_2^2$ and $\gamma_t = 2\alpha_t - \alpha_t^2$. Since $1 - \|r_t\|_2^2 \leq 1$,

$$x_{t+1} \leq (1 - \gamma_t)x_t + \frac{1}{N_t}.$$

The upper bound reduces the vector recursion to a scalar one: the previous expected squared error is contracted by $1 - \gamma_t$, while $1/N_t$ bounds the fresh sampling noise. If $\sum_t \alpha_t = \infty$ and $N_t \alpha_t$ grows without bound, then $\sum_t \gamma_t = \infty$ and $(1/N_t)/\gamma_t$ tends to zero. The two limits have complementary meanings: contraction keeps accumulating while finite-sampling noise becomes negligible relative to it. Given any $\varepsilon > 0$, eventually $1/N_t \leq \varepsilon \gamma_t$, and hence

$$x_{t+1} - \varepsilon \leq (1 - \gamma_t)(x_t - \varepsilon).$$

Once the noise is bounded by an arbitrarily small multiple of the contraction strength, subtracting ε leaves another contracting recursion. Iteration and the vanishing product $\prod_t (1 - \gamma_t)$ imply $\limsup_t x_t \leq \varepsilon$. Since ε is arbitrary, the limiting upper bound is zero and x_t tends to zero, establishing mean-square convergence. The schedule $\alpha_t = 1/(t + 2)$ and $N_t = (t + 2)^2$ therefore supplies a direct finite-sample counterexample to the claim that a vanishing grounding fraction necessarily causes collapse.